

Benchmarking LLMs for Unit Test Generation from Real-World Functions

DONG HUANG, National University of Singapore, Singapore

JIE M. ZHANG, King's College London, London, UK

MARK HARMAN, University College London, London, UK

QIANRU ZHANG*, The University of Cambridge, Cambridge, UK

MINGZHE DU*, National University of Singapore, Singapore

SEE-KIONG NG, National University of Singapore, Singapore

Recently, large language models (LLMs) have shown great promise in automating unit test generation, significantly reducing the manual effort required by developers. To effectively evaluate the capabilities of LLMs in this domain, it is crucial to have a well-designed benchmark that accurately reflects real-world scenarios and mitigates common pitfalls. Existing LLM test generation benchmarks are limited by two critical drawbacks: data contamination and structurally simple function code. As a result, scientific conclusions drawn from empirical studies using these benchmarks may be compromised. The evidence presented may be biased due to contamination and may fail to generalize beyond toy programs due to structural simplicity.

To address these problems, we introduce ULT (UnLeakedTestbench), a new benchmark specifically designed for function-level unit test generation from real-world Python functions. ULT is constructed through a multi-stage curation process that ensures high cyclomatic complexity and mitigates test case contamination. With 3,909 carefully selected function-level tasks, ULT provides a more realistic and challenging evaluation of LLMs' test generation capabilities. We also provide PLT (PreLeakedTestbench), a pair benchmark of ULT with leaked tests designed to enable a controlled analysis of memorization versus reasoning in test generation. Based on the two datasets, we conduct a large-scale empirical study involving 12 state-of-the-art LLMs, comparing their performance against established benchmarks. Our evaluation results demonstrate that ULT is significantly more challenging. For example, test cases generated by LLMs only achieve 41.32%, 45.10%, 30.22%, and 40.21% for accuracy, statement coverage, branch coverage, and mutation score on average for all LLMs, respectively. These results are substantially lower than the corresponding metrics on TestEval (91.79%, 92.18%, 82.04%, and 49.69%) and PLT (47.07%, 55.13%, 40.07%, and 50.80%).

In addition, different from existing benchmarks, ULT shows a strong correlation between test generation performance and code generation performance. For example, the correlation coefficient between the coding ability and test generation performance (*Pass@1*) on ULT is 0.79 ($p = 0.002$), while it is only 0.56 ($p = 0.059$) and 0.52 ($p = 0.080$) on TestEval and PLT, respectively. This indicates that ULT more effectively measures the generalization ability of LLMs. We also make ULT and evaluation results publicly available to foster further research¹. ULT is available at <https://github.com/huangd1999/>

*Both Qianru Zhang and Mingzhe Du are corresponding authors.

¹To preserve the integrity of the benchmark and prevent test case leakage into future LLM training sets, we do not release the ground-truth tests. Instead, we provide the complete evaluation results for all models benchmarked in this paper. This allows researchers to compare new models against our findings without compromising the benchmark.

Authors' addresses: Dong Huang, dhuang@nus.edu.sg, National University of Singapore, Singapore; Jie M. Zhang, jie.zhang@kcl.ac.uk, King's College London, London, UK; Mark Harman, mark.harman@ucl.ac.uk, University College London, London, UK; Qianru Zhang, qz348@cam.ac.uk, The University of Cambridge, Cambridge, UK; Mingzhe Du, mingzhe@nus.edu.sg, National University of Singapore, Singapore; See-Kiong Ng, seekiong@nus.edu.sg, National University of Singapore, Singapore.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, or post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM 1557-7392/2026/3-ART

<https://doi.org/10.1145/3805043>

UnLeakedTestBench.

CCS Concepts: • **Computing methodologies** → *Natural language generation*; • **Software and its engineering** → *Software defect analysis*.

Additional Key Words and Phrases: Large language models, unit test generation

1 Introduction

Reliable and robust software systems are essential in today’s technology-driven world, where software failures can lead to significant financial losses, reputational damage, and even safety risks. Effective software testing is the cornerstone of achieving this reliability, serving as a systematic approach to validate that software behaves as intended under various conditions and to identify defects before deployment [1, 2]. Among the various layers of testing, unit testing holds a particularly critical position [3]. Unit testing involves the creation of specific test inputs designed to scrutinize individual components or “units” of software, typically functions, in isolation. The primary objective of unit test generation is to cover diverse program statements and execution branches [4–6], thereby detecting defects early in the development lifecycle when they are least expensive to fix, and facilitating safer code refactoring and maintenance. However, despite its importance, the manual composition of comprehensive unit test suites is usually labor-intensive and intellectually demanding. Existing research and industry experience consistently highlight the significant manual effort, time investment, and domain expertise traditionally required, making it a frequent bottleneck in agile development environments [5, 7, 8].

To address this challenge and accelerate the unit test generation process, recent research has increasingly explored the application of LLMs in automated test case generation [9–17]. With their advanced capabilities in code understanding and generation, LLMs have shown promise in automating aspects of test case generation. For instance, LLMs have been employed to generate syntactically and semantically valid inputs for fuzzing Deep Learning (DL) libraries by implicitly learning complex API constraints (e.g., TitanFuzz [18]), and further refined to synthesize unusual programs by leveraging historical bug data (e.g., FuzzGPT [19]). Other applications include synthesizing syscall specifications for kernel fuzzing (e.g., KernelGPT [14]), employing multi-agent LLM frameworks for white-box compiler testing (e.g., WhiteFox [17]), and enhancing the general reasoning and learning capabilities of LLM agents to improve coding performance (e.g., Reflexion [13]). These diverse efforts underscore the potential of LLMs to reduce the manual effort required for test generation and enhance testing sophistication. However, the quality of LLM-generated tests is also critical; inadequately generated tests may fail to detect crucial bugs, leading to a false sense of security and the propagation of vulnerabilities into production systems. To effectively evaluate the capabilities of LLMs in generating unit tests, it is essential to have a well-designed benchmark that accurately reflects real-world scenarios and mitigates common pitfalls.

Several benchmarks have emerged to address this need [5, 6, 20, 21]. For example, TestEval [5] introduced tasks based on 210 Python programs from LeetCode, focusing on achieving overall coverage, targeted line/branch coverage, and path coverage. TestBench [21] introduced a benchmark for evaluating LLMs on test generation tasks, focusing on the ability to generate tests that cover specific statements and branches in Java classes collected from real-world projects. SWT-Bench [20] transformed code repair tasks from SWE-Bench [22] into test generation tasks for issue reproduction, evaluating generated tests on whether they fail on the original code but pass on the patched version. Similarly, TestGenEval [6] adapted SWE-Bench to create tasks for full test file generation and test completion, using execution-based metrics on code from large, well-maintained repositories. These benchmarks have made significant contributions to the evaluation of LLMs on more realistic codebases, providing valuable insights into their capabilities and limitations.

However, despite these contributions, we argue that the conclusions of prior studies rest on a precarious scientific foundation due to several critical limitations that challenge their scientific validity. Firstly, a foundational mismatch in evaluation granularity exists. In software development, unit testing is typically performed at the

function level, where individual functions are tested in isolation. However, benchmarks like TestBench [21], SWT-Bench [20], and TestGenEval [6] often operate at the class, integration, or even a file level, which does not accurately reflect common practice. Secondly, we identify the threat of insufficiently demanding evaluation (due to the “toy” code examples denoting relatively trivial test generation challenges). For scientific conclusions to be valid, they must be tested on realistic code; while it is hard to determine what constitutes “realistic” code, many existing benchmarks are clearly unrealistic due to their low structural complexity. Specifically, many of their functions exhibit low cyclomatic complexity, which can lead to inflated performance metrics as models are not adequately challenged to handle complex logic.

Finally, a more insidious threat is data leak-through. A significant concern is data contamination, where the code and associated tests from public repositories may have already been part of the training corpora for many contemporary LLMs² [23, 24]. This leakage compromises the reliability of an evaluation, as a model might achieve high performance due to memorization rather than genuine capability. These combined limitations mean that previous work may be scientifically unreliable, potentially presenting a distorted view of LLM capabilities.

This paper directly and systematically confronts these issues by introducing ULT (UnLeakedTestBench), a new benchmark that offers a more sound scientific basis on which to base future empirical evaluations of LLM-based unit test generation. The construction of ULT was guided by three key principles: real-world relevance, cyclomatic complexity, and decontamination. Specifically, we sourced our candidate functions from The Stack v2 [25], a large and diverse corpus of permissively licensed source code. To ensure the functions in ULT are sufficiently complex, we filtered out those with low cyclomatic complexity, retaining only those that present a meaningful challenge. Most importantly, to mitigate data contamination, we rigorously filtered out functions that have associated test cases already present in the training corpus. This process yielded a total of 3,909 function-level tasks, each with a cyclomatic complexity of at least 10, ensuring that the functions challenge an LLM’s reasoning ability rather than its memorization.

To enable a controlled study of data contamination’s effects, we also introduce a counterpart benchmark: PLT (PreLeakedTestBench). This benchmark is composed of the very functions that were excluded during the decontamination phase of creating ULT. These functions also meet our high-complexity criteria but are intentionally “leaked”, as their ground-truth tests were found within the public data. By providing this set alongside its clean counterpart, we facilitate a direct and controlled analysis of memorization versus reasoning. Using ULT and PLT in tandem allows researchers to accurately measure performance inflation and achieve a more transparent and scientifically valid assessment of LLM capabilities in unit test generation. However, we acknowledge the inherent limitations of this observational comparison. While we align the two benchmarks on metrics like cyclomatic complexity, other confounding factors may remain. For instance, code with existing tests (the source of PLT) typically originates from well-maintained repositories, which may also feature higher-quality documentation or cleaner coding styles compared to code without tests (ULT). Consequently, performance discrepancies should be interpreted with these potential confounding variables in mind, rather than attributed exclusively to data leakage.

To evaluate the effectiveness of ULT, we conducted a large-scale empirical study involving 12 state-of-the-art LLMs, comparing their performance on ULT against established benchmarks such as TestEval [5], as well as the counterpart benchmark PLT. Our experiments reveal that ULT poses a significantly greater challenge than other benchmarks, with LLMs achieving substantially lower accuracy and code coverage. For example, test cases generated by LLMs only achieve 41.32%, 45.10%, 30.22%, and 40.21% for accuracy, statement coverage, branch coverage, and mutation score on average for all LLMs, respectively, which are substantially lower than the corresponding metrics on TestEval (91.79%, 92.18%, 82.04%, and 49.69%) and PLT (47.07%, 55.13%, 40.07%, and 50.80%). Moreover, we observe a strong correlation between test generation performance and the code generation

²<https://conf.researchr.org/details/icse-2025/icse-2025-SRC/6/Revisiting-SWE-Bench-On-the-Importance-of-Data-Quality-for-LLM-based-Code-Models>

performance of LLMs on ULT. For example, the correlation coefficient between the code generation performance and test generation performance (*Pass@1*) on ULT is 0.79 ($p = 0.002$), while it is only 0.56 ($p = 0.059$) and 0.52 ($p = 0.080$) on TestEval and PLT, respectively. This finding underscores the effectiveness of ULT in measuring the true test generation capabilities of LLMs, as it mitigates the influence of memorization and focuses on the model’s ability to reason about and generate tests for complex, real-world code.

In summary, this paper makes the following contributions:

- We present ULT, a new, large-scale benchmark for evaluating LLM-based unit test generation, comprising 3,909 real-world Python functions. Its key innovation lies in a rigorous curation process that ensures high cyclomatic complexity and mitigates test case contamination. Our experiments demonstrate that ULT poses a significantly greater challenge than existing benchmarks, revealing the current limitations of LLMs when faced with complex and uncontaminated real-world code.
- We also present PLT, a counterpart benchmark designed to isolate the effects of data contamination while ensuring tasks are both realistic and complex. It allows for a controlled analysis of memorization versus reasoning in test generation, enabling researchers to accurately measure performance inflation and achieve a more transparent and scientifically valid assessment of LLM capabilities.
- We conduct a large-scale empirical study involving 12 state-of-the-art LLMs, comparing their performance on ULT against established benchmarks such as TestEval and PLT. Our results reveal that ULT presents a significantly greater challenge, with LLMs achieving substantially lower accuracy and code coverage compared to existing benchmarks.
- We provide strong empirical evidence that our design principles lead to a more reliable evaluation. We show that performance on ULT has a strong, positive correlation with a model’s intrinsic coding ability, confirming that it measures generalization. Conversely, we demonstrate that performance on contaminated data is skewed by memorization, particularly for branch coverage.

2 Background

In this section, we provide the necessary background to understand the context and significance of our work. We begin by discussing the importance of software testing, particularly unit testing, in ensuring software quality and reliability. We then explore the historical evolution of automated test case generation techniques, highlighting the challenges faced by traditional methods. Finally, we introduce the recent advancements in LLMs and their potential to revolutionize test case generation, while emphasizing the critical need for rigorous benchmarks to evaluate their effectiveness.

2.1 Unit Testing in Software Engineering

In recent years, software systems have been widely adopted across various domains, including web applications, mobile apps, embedded systems, and enterprise software. These systems exist in nearly every aspect of daily life and commerce, making their reliability and robustness paramount [26]. If the software systems fail, they can lead to significant consequences, including financial losses, reputational damage, and even risks to human safety. To ensure software quality, software testing is the primary mechanism employed within the discipline of software engineering [27]. It encompasses a systematic collection of activities designed to verify that a software system performs according to its specifications and to identify defects prior to operational deployment. Software testing is typically structured into several distinct levels, such as unit testing, integration testing, system testing, and acceptance testing, each with a specific focus and scope, where unit testing occupies a foundational position in the development lifecycle [28]. The primary goal of unit testing is to validate that each unit behaves as expected under various conditions, thereby ensuring that the individual components function correctly before they are integrated into larger systems [29]. Effective unit testing can significantly reduce the number of defects that

propagate to later stages of development, where they become more costly and complex to resolve [27].

2.2 Automated Test Case Generation

Despite unit testing can substantially improve software quality, it still poses significant challenges for software developers, i.e., the manual creation of comprehensive test suites is often labor-intensive, time-consuming, and requires a deep understanding of both the code under test and established software testing principles [7]. The key reason is that crafting effective test cases necessitates meticulous consideration of a wide array of input values, including typical cases, boundary conditions, and potential edge cases, to ensure adequate exploration of the unit's functionality and execution paths. If the development team is under tight deadlines, they may not have sufficient time to create comprehensive test suites, leading to insufficient coverage and potentially undetected bugs. Then, the manual efforts required by the test generation become a bottleneck in many software development workflows, as developers must balance the need for thorough testing with the constraints of time and resources [30]. This challenge is further compounded by the increasing complexity of modern software systems, which often involve intricate logic, numerous dependencies, and diverse input formats. To address these challenges, the field of software engineering has long pursued the development of automated test case generation (ATG) techniques. Traditional ATG methods, such as symbolic execution [31, 32], search-based software testing (SBST) [33], and fuzz testing [34], have been widely incorporated into development tools. However, they are often confronted with inherent limitations, such as scalability issues (e.g., path explosion in symbolic execution) and difficulty in handling complex program states or generating semantically relevant inputs [1]. Unguided random fuzzing, for instance, may be inefficient in achieving deep coverage of intricate program logic without specific domain knowledge.

2.3 LLM-Driven Automated Test Generation

To address the limitations of traditional ATG methods, recent research has increasingly turned to utilizing LLMs for automated test generation [9, 11, 14, 17–19, 35]. Different with traditional ATG techniques, LLMs leverage their extensive training on diverse codebases and natural language to generate test cases that are not only syntactically correct but also semantically meaningful and contextually relevant. Compared to traditional ATG methods, existing works in LLM-driven test generation have demonstrated significant promise and versatility across diverse testing scenarios. For example, TitanFuzz [18] employs LLMs to generate syntactically and semantically valid inputs for fuzzing Deep Learning (FDL) libraries, implicitly learning complex API constraints, while FuzzGPT [19] refines LLMs to synthesize unusual programs for fuzzing by leveraging historical bug data. Beyond input generation, LLMs are being explored for more complex testing workflows, including the synthesis of syscall specifications for kernel fuzzing (e.g., KernelGPT [14]) and the development of multi-agent LLM frameworks for white-box compiler testing (e.g., WhiteFox [17]). Another notable application includes the generation of test cases for specific functions or classes, where LLMs can produce syntactically valid inputs that cover various execution paths and edge cases. For instance, CodeT [9] and AgentCoder [11] utilize LLMs to generate test cases for specific functions, while Mercury [12] employs LLMs to generate test cases for complex Python functions with intricate logic. These approaches have demonstrated the potential of LLMs to significantly reduce manual effort, improve the quality and relevance of generated tests, and handle complex code structures with greater intuitive facility than some traditional methods. However, despite the promising results, the effectiveness of LLMs in test generation is not uniform and can vary significantly based on several factors, including the model architecture, training data, and specific task formulation.

2.4 The Need for Rigorous Benchmarking

To effectively evaluate the capabilities of LLMs in generating unit tests, it is essential to have a well-designed benchmark that accurately reflects real-world scenarios and mitigates common pitfalls. A well-constructed benchmark for LLM-based test generation should ideally provide a multi-dimensional assessment. This includes

evaluating their ability to generate correct test cases, the extent to which these tests cover the code under test (e.g., statement and branch coverage), and their effectiveness in detecting faults through metrics such as mutation testing scores [5]. Recently, several benchmarks have emerged to address this need [5, 6, 20, 21]. For example, TestEval [5] introduced tasks based on 210 Python programs from LeetCode, focusing on achieving overall coverage, targeted line/branch coverage, and path coverage. TestBench [21] introduced a benchmark for evaluating LLMs on test generation tasks, focusing on the ability to generate tests that cover specific statements and branches in Java classes collected from real-world projects. SWT-Bench [20] transformed code repair tasks from SWE-Bench [22] into test generation tasks for issue reproduction, evaluating generated tests on whether they fail on the original code but pass on the patched version. Similarly, TestGenEval [6] adapted SWE-Bench to create tasks for full test file generation and test completion, using execution-based metrics on code from large, well-maintained repositories. These benchmarks have made significant contributions to the evaluation of LLMs on more realistic codebases, providing valuable insights into their capabilities and limitations.

However, despite their contributions, existing benchmarks face several critical limitations, which can undermine the reliability and generalizability of their findings. Firstly, in software development, unit testing is typically performed at the function level, where individual functions are tested in isolation. However, benchmarks like TestBench, SWT-Bench, and TestGenEval often operate at the class, integration, or even a file level, which may not accurately reflect the common practice of unit testing at the function level. Secondly, many of the functions in these benchmarks exhibit low cyclomatic complexity, meaning they are structurally simple and do not adequately challenge an LLM’s ability to generate tests for more complex logic and diverse execution paths. This can lead to inflated performance metrics, as LLMs can achieve high coverage with relatively few, straightforward test cases. Finally, a significant concern is data contamination, where the code and potentially associated tests from public repositories may have already been part of the training corpora for many contemporary LLMs [23]. This leakage compromises the reliability of the evaluation, as LLMs might demonstrate high performance due to memorization rather than genuine test generation capabilities. These limitations can lead to an unreliable assessment of an LLM’s inherent test generation ability, as the benchmarks may not accurately reflect the challenges and complexities encountered in real-world software development.

To address these limitations and provide a more reliable benchmark, we introduce ULT, a novel dataset constructed for evaluating LLM-driven test case generation for real-world, function-level Python tasks. ULT is designed to mitigate the identified shortcomings of existing benchmarks by focusing on function-level testing, incorporating a diverse set of tasks with varying complexity, and ensuring that the dataset is free from data contamination. By providing a more accurate and challenging evaluation framework, ULT aims to enhance the understanding of LLM capabilities in generating high-quality unit tests for Python code.

3 Methodology

3.1 Overview

In this section, we present the benchmark construction process for ULT and PLT, two benchmarks designed to evaluate LLM-driven unit test generation for real-world Python functions. Specifically, we first detail the multi-stage curation process designed to yield a high-quality dataset characterized by real-world relevance, controlled complexity, and mitigated data contamination. Subsequently, we define the distinct task formats through which LLMs are prompted and their test generation capabilities are assessed. Finally, we outline the specific effectiveness metrics used to quantify and compare the performance of LLMs in these tasks.

3.2 Benchmark Construction

Data Collection We aim to evaluate LLMs’ ability to generate unit tests for real-world Python functions, focusing on their capacity to cover diverse program statements and execution branches. To achieve this, we begin by

collecting candidate Python functions from The Stack v2 [25], a large and diverse corpus of permissively licensed source code. The Stack v2 is a vast collection of open-source code, encompassing a wide range of programming languages and domains, making it an ideal source for our benchmark. After collecting the candidate functions, we implement a rigorous multi-stage filtering pipeline to select suitable candidates. The filtering process is designed to ensure that the selected functions are sufficiently complex, self-contained, and free from contamination (for ULT only), by pre-existing test cases. We introduce each step in the following.

Filtering by Cyclomatic Complexity One of the core aims of our benchmark work is to assess an LLM’s ability to generate test cases for functions with non-trivial control flow, thereby requiring tests that cover code statements and branches. Functions that are overly simplistic (e.g., possessing only a single execution path) do not offer a discerning challenge for this purpose. To address this, we follow the setup of TestEval [5] and employ cyclomatic complexity³ as a quantitative measure of a program’s structural complexity.

Cyclomatic complexity measures the number of linearly independent paths through a program’s source code. It acts as a proxy for the logical “difficulty” of a function: a higher value implies the function contains more decision points (such as “if” statements, loops, and exception handlers), making it structurally intricate. We use this metric to ensure we are not evaluating models on trivial code, but rather on functions that require sophisticated reasoning to test comprehensively. Given the control flow graph of a program, its cyclomatic complexity V is defined as $V = e - n + p$, where e is the number of edges, n is the number of nodes, and p is the number of connected components in the graph. A higher cyclomatic complexity generally indicates a greater number of decision points and potential execution paths within a function. We filter out functions with a cyclomatic complexity of less than 10. The functions retained in our dataset after all filtering stages possess an average cyclomatic complexity of 14.87, ensuring a substantive level of logical complexity for testing.

We acknowledge that the use of cyclomatic complexity can be controversial, with some studies arguing that it often serves as a proxy for code size [36, 37]. However, we do not claim it necessarily represents “conceptual complexity.” Instead, we use it strictly as a structural filter to guarantee the selection of functions with nested Abstract Syntax Tree (AST) structures that characterize complex control flow. Without this filter, we might inadvertently include long methods that lack any significant branching substructure. By enforcing a minimum cyclomatic complexity, we explicitly select for functions that possess non-trivial branching logic. Therefore, we use it as a measure of AST complexity, and since it is also correlated with size, we ensure that as complexity increases, we are evaluating larger programs with meaningful branching. We further validate this assumption by analyzing the correlation between cyclomatic complexity and the performance of LLMs on our test generation tasks (See Section 5.2).

Self-Containment For effective unit test generation, it is crucial that the function under test (FUT) can be evaluated in relative isolation, without complex external dependencies that the LLM might lack context for, such as custom functions or classes defined elsewhere in a large project. If a function heavily relies on other custom functions or classes defined elsewhere in a large project, an LLM provided only with the FUT’s source code may struggle to generate correct and executable tests due to missing knowledge of these external components. Therefore, we analyzed the remaining functions to identify and filter out those that exhibited direct dependencies on or interactions with other non-standard, user-defined functions or complex class structures not passed as simple arguments. This step aims to ensure that the context provided to the LLM (primarily the function’s own source code and potentially its signature) is largely sufficient for generating meaningful tests. This design choice aligns with the fundamental principle of unit testing, which seeks to verify internal logic in isolation. By filtering out deep cross-module dependencies, we specifically evaluate the model’s ability to generate logic-covering inputs without the confounding risk of hallucinating external state or dependencies.

Testability Guarantee While a function might be self-contained in terms of custom project-specific code, it often

³https://en.wikipedia.org/wiki/Cyclomatic_complexity

relies on standard Python libraries or widely-used third-party packages (e.g., numpy, pandas, glob, csv, logging). For each of the functions that passed the cyclomatic complexity and self-containment filters, we first verified that it was indeed testable by prompting GPT-4o to generate at least three distinct, executable test inputs. If a function could not be tested by the input tests, we then feed it to GPT-4o to check whether bugs exist in the function. If bugs are found, we will require GPT-4o to fix the bugs and re-verify the testability of the function. We set the debugging limit to 3 times, i.e., if the function is still not testable after 3 times of debugging, we will discard the function. This step served as a crucial quality check, ensuring that the functions selected for the benchmark are available to unit testing. During this process, we also identified and explicitly added necessary import statements for standard Python libraries and common, permissively licensed third-party libraries directly into the context provided for each function. This ensures that the code, when presented to an LLM and subsequently when its generated tests are executed, has its immediate dependencies readily available.

Decontamination The final step in our benchmark construction is the decontamination process, which allows us to define two benchmarks for a controlled analysis: ULT and PLT. To put it simply, this step is designed to distinguish between a model’s ability to *reason* versus its ability to *memorize*. If an LLM has seen the specific test cases for a function during its pre-training (effectively having seen the “answer key” to the test generation exam), it might achieve high scores by simply recalling the test rather than generating it through analysis. To prevent this “cheating” and measure true generalization, we must remove functions whose answers are already public knowledge. To achieve this, we first extracted the name of each candidate function (e.g., `func_name`) and searched The Stack V2 for corresponding test definitions (e.g., `def test_func_name`) or assertions (e.g., `assert func_name`). Functions for which no corresponding test cases were found form our primary benchmark, ULT, providing a rigorous evaluation of an LLM’s generalization ability. Our second benchmark, PLT, is a superset that contains all the functions from ULT plus all the functions that were identified as potentially contaminated. By including the decontaminated set within the leaked set, PLT represents a benchmark with mixed data quality, allowing researchers to measure the specific impact of data contamination by comparing performance against the purely decontaminated ULT.

After applying these steps, we retained a total of 3,909 function-level tasks for ULT, and 18,169 function-level tasks for PLT.

3.3 Task Definition

To measure the test generation capabilities of LLMs, we follow the setup of TestEval [5] and define a K -query function-level unit test generation task. This task is designed to evaluate the model’s ability to generate high-quality (i.e., correct, diverse, and effective) test cases for a given function under test (FUT). During the evaluation, the LLM is first provided with the source code of a single function and is prompted to generate a set of test cases that cover various aspects of the function’s behavior. Then, for other queries ($2 \leq i \leq K$), the LLM is provided with the source code of the FUT and all previously generated test cases, and is prompted to generate a new test case that is distinct from all previously generated ones. The detailed task definition is as follows:

- **Initial Round (Round 1):** In the first round ($i = 1$), the LLM is provided with the source code of the FUT. It is then prompted to generate a single unit test case for this function.
- **Subsequent Rounds (Round $i > 1$):** For each subsequent round i (from 2 to K), the prompt is dynamically updated and expanded. The new prompt contains both the original FUT and the set of all test cases that were successfully generated in the previous rounds ($1, \dots, i - 1$). The LLM is then explicitly instructed to generate one new test case that is distinct and different from all the previously generated ones provided in the prompt’s context.

This iterative process is repeated until K test cases have been generated. The final output is a test suite built through a sequence of diversification requests.

3.4 Effectiveness Metrics

We employ a suite of well-established software testing metrics to measure the quality of the LLM-generated test cases, which are designed to assess various aspects of the generated tests, including their correctness, coverage, and fault-detection capabilities.

3.4.1 Test Generation Accuracy ($Pass@k$). Test generation accuracy measures the proportion of correct test cases generated by the LLM. A test is treated as “correct” if it is **syntactically valid** (i.e., it does not have syntax errors), runs to completion, and its assertions reflect valid expectations of the function’s behavior for the given inputs. In our benchmark, we define the accuracy metric as $Pass@k$, which represents the average number of correct tests generated per function, normalized by the total number of test cases requested (K). The calculation of $Pass@k$ is based on the number of correct tests generated for each function, where C_i is the number of correct tests generated for the i -th function (where $0 \leq C_i \leq K$). If N is the total number of functions in the benchmark, the accuracy is computed as follows:

$$Pass@k = \frac{\sum_{i=1}^N C_i}{N \times K} \quad (1)$$

3.4.2 Code Coverage ($LCov@k$ / $BCov@k$). In addition to accuracy, we also measure the code coverage achieved by the generated test cases. Code coverage is an important metric in software testing, as it indicates the extent to which the generated tests exercise the code under test. We report the code coverage across all functions for two standard metrics: Line Coverage ($LCov@k$) and Branch Coverage ($BCov@k$). $LCov@k$ is the percentage of executable lines in the function’s source code that are executed by at first k queries generated test cases. $BCov@k$ is the percentage of possible execution branches (e.g., from “if” or “while” statements) that are traversed by the test suite generated by the first k queries. We also provide the improvement of coverage over the first query, i.e., $\Delta_{LCov@k}$ and $\Delta_{BCov@k}$, to measure the incremental coverage achieved by each additional test case generated.

3.4.3 Mutation Score ($Mut@k$). Finally, we follow the existing work [6] and measure the fault-detection capability of the generated test cases using mutation score. Mutation score is a widely used metric in software testing that evaluates the effectiveness of a test suite in detecting faults [38, 39]. This metric involves automatically introducing small, syntactic changes (e.g., changing “+” to “-”, or “>” to “>=”) into the function under test to create faulty versions known as “mutants”. In our evaluation, we use Cosmic-Ray⁴ to generate all possible mutants for each function in ULT. A test suite “kills” a mutant if it fails when run against the mutated code but passes on the original. The mutation score is the percentage of non-equivalent mutants that are killed by the generated test suite. In our evaluation, for each function, we set timeout = 120 to avoid the test suite running indefinitely on a mutant. The mutation score is calculated as follows:

$$Mut@k = \frac{\sum_{i=1}^N M_i}{N} \quad (2)$$

where M_i is the number of mutants killed by the test suite generated for the i -th function, and N is the total number of functions in the benchmark. A higher score indicates a more effective test suite at finding bugs.

It is also important to clarify our evaluation methodology regarding subsequent metrics: for the Code Coverage (Tab. 4, Tab. 5) and Mutation Score (Tab. 6) reported in this paper, we strictly consider only *accurate* test cases (i.e., those that satisfy assertions). This ensures that our effectiveness metrics reflect the capability of valid, correct tests rather than inflating scores with executable but logically flawed code.

⁴<https://cosmic-ray.readthedocs.io/en/latest/>

4 Experiment Design

4.1 Research Questions

We formulate three research questions that guide our empirical evaluation. These questions are designed to compare the performance of state-of-the-art LLMs on ULT against established benchmarks as well as PLT.

- **RQ1: To what extent do LLMs struggle with test generation on ULT compared to other benchmarks?**
- **RQ2: To what extent does the cyclomatic complexity of functions in ULT influence LLM performance compared to other benchmarks?**
- **RQ3: How does data contamination affect the assessment of test case generation?**

4.2 Models and Baselines

To answer our research questions, we conduct a comprehensive study involving a diverse set of LLMs and a comparative analysis against established benchmarks.

4.2.1 Evaluation LLMs. We selected a wide range of 12 state-of-the-art LLMs to ensure a broad and representative evaluation. The selection includes models of varying sizes, from smaller models with under 2 billion parameters to large models with over 30 billion parameters. It also covers both general-purpose instruction-tuned models and models specifically specialized for code generation and understanding. The complete list of evaluated models is detailed in Tab. 1.

Table 1. The set of LLMs evaluated in our study, categorized by their accessibility and originating developer.

Category	Models
<i>CodeLlama</i>	CodeLlama-7b-Instruct-hf
<i>Seed-Coder</i>	Seed-Coder-8B-Instruct
<i>DeepSeekCoder</i>	deepseek-coder-1.3b-instruct, 6.7b-instruct, 33b-instruct
<i>Gemma-3</i>	gemma-3-4b-it, 12b-it, 27b-it
<i>Qwen2.5-Coder</i>	Qwen2.5-Coder-7B-Instruct, 14B-Instruct, 32B-Instruct
<i>Microsoft Phi-4</i>	Phi-4-mini-instruct

4.2.2 Baseline Benchmarks. Several benchmarks have been proposed to measure the capability of LLMs in generating test cases, such as TestEval [5], TestGenEval [6], TestBench [21], and SWT-Bench [20]. However, these benchmarks either focus on programming languages other than Python (e.g., TestBench focuses on Java tasks) or do not specifically target function-level unit test case generation (e.g., TestGenEval and SWT-Bench). In this work, we then focus on the performance of the selected LLMs on ULT against TestEval [5]. TestEval is a well-established benchmark for evaluating LLMs on test generation tasks, focusing on Python functions from competitive programming-style problems. Similar to ULT, the tasks in TestEval are also designed to the function-level unit test generation, making it a suitable baseline for our study. In addition, we also include PLT, which is a superset of ULT that includes all the functions from ULT plus all the functions that were identified as potentially contaminated. This allows us to measure the specific impact of data contamination by comparing performance against the purely decontaminated ULT.

4.3 Inference Configuration

To ensure the fairness of our evaluation and the reproducibility of our results, we employ a consistent inference configuration across all evaluated LLMs. For generating test cases, we use a greedy decoding strategy to minimize randomness and ensure that the output is primarily a function of the model’s inherent capabilities rather than stochastic sampling. Specifically, we set the decoding temperature to 0.0. This low value encourages the model to select high-probability tokens, yielding outputs that are stable and deterministic while still allowing for some minor variation from a purely greedy approach. The maximum number of new tokens to be generated for any given task is set to 1024, which is sufficient to generate test cases without truncation.

5 Results and Findings

5.1 RQ1: To what extent do LLMs struggle with test generation on ULT compared to other benchmarks?

To answer RQ1, we compare the performance of various LLMs on ULT against their performance on TestEval and PLT⁵.

5.1.1 RQ1.1: Accuracy and Executability of LLM-generated test cases. We first analyze the accuracy of the test cases generated by the LLMs on ULT, PLT, and TestEval. The evaluation results of LLM-generated test cases are shown in Tab. 2, which demonstrate that the accuracy of LLM-generated test cases on ULT is significantly lower than on TestEval and PLT. For example, the average *Pass@1* across all models is 12.69% on ULT, compared to 48.42% on PLT and 57.74% on TestEval. The *Pass@2* and *Pass@5* metrics show similar trends, with average scores of 12.60% and 12.57% on ULT, respectively, compared to 46.54% and 44.45% on PLT, and 55.24% and 51.93% on TestEval.

To provide a deeper insight into the failure modes of LLMs, we also evaluated the *Executability Rate* of the generated test cases, which defined as the percentage of tests that are syntactically correct and run without raising runtime exceptions (ignoring assertion failures). The results are presented in Tab. 3. We observe that executability is consistently high across all benchmarks, with an overall average of 84.75% on ULT. This indicates that modern LLMs are highly proficient at generating syntactically valid Python code. However, the stark contrast between the high executability (~85%) and the low accuracy (~12%) on ULT highlights that the primary challenge is not code generation itself, but the logical reasoning required to construct tests that correctly capture the function’s behavior (i.e., passing assertions).

The results indicate that LLMs struggle significantly with test generation on ULT compared to PLT and TestEval, suggesting that ULT presents a more challenging and realistic evaluation of LLMs’ test generation capabilities compared to other benchmarks.

*Answer to RQ1.1: LLMs struggle significantly with generating correct tests on ULT compared to other benchmarks. The average *Pass@5* score on ULT is 12.57%, while it is 44.45% on PLT and 51.93% on TestEval. This suggests that ULT presents a more challenging evaluation of LLMs’ test generation capabilities.*

5.1.2 RQ1.2: Line coverage of LLM-generated test cases. In addition to accuracy, we also evaluate the line coverage of the test cases generated by the LLMs in ULT, PLT, and TestEval. The code line coverage results of LLM-generated test cases are shown in Tab. 4. We can observe that the average line coverage (*LCov@k*) of LLM-generated test cases on ULT is lower than on TestEval and PLT across all models and all *k* values. For example, the average *LCov@1* across all models is 36.22% on ULT, while it is 39.62% on PLT and 87.81% on TestEval, when the *K* is

⁵We also provide the cyclomatic complexity distribution level analysis in Section 5.2 and Section 5.3.1 to further analyze the performance of LLMs.

Table 2. RQ1.1: Accuracy of LLM-generated test cases in TestEval, ULT, and PLT. $Pass@k$ represents the percentage of the first K queries generated test cases that correctly pass the function under test. The values in parentheses indicate the improvement over the $Pass@1$. **Insight:** LLMs achieve significantly lower accuracy on ULT (Avg 12.69%) compared to PLT (Avg 48.42%) and TestEval (Avg 57.74%), indicating that uncontaminated, complex tasks present a much harder challenge.

Model_Name	Pass@1			Pass@2			Pass@5		
	ULT	PLT	TestEval	ULT	PLT	TestEval	ULT	PLT	TestEval
CodeLlama-7b-Instruct-hf	9.98	40.99	68.10	9.68 (-0.29)	37.28 (-3.71)	66.67 (-1.43)	8.78 (-1.19)	32.72 (-8.27)	66.10 (-2.00)
Seed-Coder-8B-Instruct	13.28	52.70	76.67	14.26 (+0.98)	51.06 (-1.64)	74.29 (-2.38)	14.32 (+1.04)	48.94 (-3.75)	68.29 (-8.38)
gemma-3-4b-it	11.03	40.52	30.00	11.33 (+0.31)	40.18 (-0.34)	29.05 (-0.95)	11.78 (+0.75)	39.35 (-1.18)	30.48 (+0.48)
gemma-3-12b-it	12.38	51.40	52.86	12.48 (+0.10)	49.96 (-1.44)	55.24 (+2.38)	13.69 (+1.31)	48.84 (-2.56)	54.29 (+1.43)
gemma-3-27b-it	17.83	53.12	59.05	20.26 (+2.43)	53.89 (+0.78)	60.95 (+1.90)	21.64 (+3.81)	54.00 (+0.89)	60.76 (+1.71)
Qwen2.5-Coder-7B-Instruct	12.48	52.54	52.38	13.47 (+0.98)	51.23 (-1.32)	57.14 (+4.76)	14.05 (+1.57)	49.30 (-3.25)	53.33 (+0.95)
Qwen2.5-Coder-14B-Instruct	15.32	57.92	70.00	15.73 (+0.41)	56.41 (-1.51)	61.90 (-8.10)	15.80 (+0.48)	54.86 (-3.07)	55.81 (-14.19)
Qwen2.5-Coder-32B-Instruct	17.83	57.38	77.62	15.90 (-1.93)	55.42 (-1.96)	66.19 (-11.43)	16.04 (-1.79)	53.95 (-3.44)	59.05 (-18.57)
deepseek-coder-1.3b-instruct	9.39	37.75	38.57	7.57 (-1.82)	34.52 (-3.23)	39.05 (+0.48)	6.56 (-2.83)	32.73 (-5.03)	39.90 (+1.33)
deepseek-coder-6.7b-instruct	10.62	44.84	55.71	9.38 (-1.24)	41.23 (-3.61)	52.62 (-3.10)	8.60 (-2.02)	37.48 (-7.36)	49.81 (-5.90)
deepseek-coder-33b-instruct	13.64	49.17	72.86	11.91 (-1.73)	46.13 (-3.04)	61.19 (-11.67)	11.02 (-2.61)	42.93 (-6.23)	56.10 (-16.76)
Phi-4-mini-instruct	8.52	42.76	39.05	9.21 (+0.69)	41.18 (-1.58)	38.57 (-0.48)	8.55 (+0.03)	38.33 (-4.43)	29.24 (-9.81)
Overall	12.69	48.42	57.74	12.60 (-0.09)	46.54 (-1.88)	55.24 (-2.50)	12.57 (-0.12)	44.45 (-3.97)	51.93 (-5.81)

Table 3. RQ1.1: Executability Rate of LLM-generated test cases. This metric measures the percentage of tests that run without runtime errors (excluding assertion errors). **Insight:** High executability rates (~85% on ULT) indicate proficiency in generating valid code. However, the sharp drop to ~12% accuracy (Tab. 2) indicates that the primary bottleneck is logical correctness rather than syntax.

Model_Name	ULT			PLT			TestEval		
	Num = 1	Num = 2	Num = 5	Num = 1	Num = 2	Num = 5	Num = 1	Num = 2	Num = 5
CodeLlama-7b-Instruct-hf	88.62	85.26	81.33	92.81	90.04	86.13	89.05	89.05	89.05
Seed-Coder-8B-Instruct	90.46	87.54	83.06	92.48	90.75	88.16	99.05	99.05	99.05
gemma-3-4b-it	91.56	89.49	87.31	93.02	91.32	89.77	80.48	80.48	80.48
gemma-3-12b-it	90.07	88.31	85.47	92.67	90.64	88.73	96.67	96.67	96.67
gemma-3-27b-it	87.93	85.52	82.71	91.87	90.13	88.22	97.62	97.62	97.62
Qwen2.5-Coder-7B-Instruct	92.71	91.51	89.18	94.05	93.11	91.69	100.0	100.0	100.0
Qwen2.5-Coder-14B-Instruct	91.51	90.2	88.26	93.49	92.21	90.94	99.52	99.52	99.52
Qwen2.5-Coder-32B-Instruct	91.51	89.08	87.11	92.73	90.96	89.6	100.0	100.0	100.0
deepseek-coder-1.3b-instruct	74.6	72.14	71.6	83.78	81.4	81.05	95.71	95.71	95.71
deepseek-coder-6.7b-instruct	61.4	56.02	51.73	74.65	71.17	67.77	98.57	98.57	98.57
deepseek-coder-33b-instruct	68.36	63.16	57.43	79.14	75.47	71.32	100.0	100.0	100.0
Phi-4-mini-instruct	88.23	85.34	80.1	89.47	87.36	83.3	80.95	80.95	80.95
Overall	84.75	81.96	78.77	89.18	87.05	84.72	94.8	94.8	94.8

set to 1. This indicates that the functions in ULT are inherently more difficult to cover, as the LLMs struggle to generate test cases that cover a significant portion of the code lines. In addition, we can also observe that when we increases the K , the $LCov@k$ of LLM-generated test cases on ULT increases, but it remains lower than on TestEval and PLT. For instance, the average $LCov@2$ on ULT is 40.43%, while it is 46.78% on PLT and 90.43% on TestEval. Similarly, the average $LCov@5$ on ULT is 45.10%, while it is 55.13% on PLT and 92.18% on TestEval. This indicates that even with multiple attempts, LLMs struggle to explore the source code of the complex, real-world functions in ULT as effectively as they do for the algorithmic problems in TestEval.

Furthermore, we can observe that the improvement in line coverage of LLM-generated test cases on ULT is lower than PLT. For example, the average improvement of line coverage on ULT is 4.22% and 8.89%, while it is

Table 4. RQ1.2: Code line coverage of LLM-generated test cases in three datasets. $LCov@k$ represents the percentage of executable lines in the function’s source code that are executed by the first K queries generated test cases. The values in parentheses indicate the improvement over the $LCov@1$. **Insight:** Line coverage on ULT (45.10% at $k = 5$) is substantially lower than on TestEval (92.18%), confirming that functions in ULT have more complex structures that are harder to cover.

Model_Name	$LCov@1$			$LCov@2$			$LCov@5$		
	ULT	PLT	TestEval	ULT	PLT	TestEval	ULT	PLT	TestEval
CodeLlama-7b-Instruct-hf	34.31	37.40	83.34	37.37 (+3.05)	43.00 (+5.61)	85.32 (+1.98)	39.67 (+5.36)	47.10 (+9.71)	86.85 (+3.52)
Seed-Coder-8B-Instruct	41.38	43.95	92.51	46.11 (+4.73)	51.54 (+7.59)	94.71 (+2.20)	51.57 (+10.19)	61.35 (+17.40)	96.60 (+4.09)
gemma-3-4b-it	35.02	38.66	71.09	38.90 (+3.87)	45.55 (+6.89)	73.35 (+2.26)	43.92 (+8.90)	54.74 (+16.08)	74.78 (+3.70)
gemma-3-12b-it	40.30	43.22	90.46	45.64 (+5.34)	52.02 (+8.80)	94.53 (+4.07)	52.05 (+11.76)	63.03 (+19.81)	97.31 (+6.85)
gemma-3-27b-it	41.09	44.75	91.13	46.76 (+5.67)	53.16 (+8.40)	94.92 (+3.80)	53.74 (+12.65)	64.03 (+19.28)	97.14 (+6.01)
Qwen2.5-Coder-7B-Instruct	40.73	43.37	91.47	45.74 (+5.01)	51.39 (+8.03)	94.85 (+3.38)	51.99 (+11.27)	62.42 (+19.05)	97.25 (+5.78)
Qwen2.5-Coder-14B-Instruct	40.64	43.39	92.35	45.44 (+4.80)	51.10 (+7.71)	94.99 (+2.63)	52.11 (+11.47)	62.89 (+19.50)	97.17 (+4.82)
Qwen2.5-Coder-32B-Instruct	41.32	44.38	91.79	47.04 (+5.72)	53.23 (+8.85)	95.12 (+3.32)	52.87 (+11.55)	63.89 (+19.51)	97.24 (+5.45)
deepseek-coder-1.3b-instruct	28.28	32.35	86.19	30.67 (+2.39)	37.37 (+5.02)	88.18 (+1.99)	30.98 (+2.70)	38.12 (+5.77)	88.24 (+2.05)
deepseek-coder-6.7b-instruct	25.88	31.72	90.46	29.05 (+3.17)	38.48 (+6.75)	93.12 (+2.66)	32.83 (+6.96)	46.86 (+15.14)	95.10 (+4.64)
deepseek-coder-33b-instruct	29.95	34.34	92.31	33.47 (+3.52)	40.93 (+6.59)	94.46 (+2.15)	37.11 (+7.17)	47.86 (+13.52)	95.75 (+3.44)
Phi-4-mini-instruct	35.69	37.86	80.63	39.02 (+3.33)	43.54 (+5.68)	81.62 (+1.00)	42.35 (+6.66)	49.32 (+11.46)	82.69 (+2.06)
Overall	36.22	39.62	87.81	40.43 (+4.22)	46.78 (+7.16)	90.43 (+2.62)	45.10 (+8.89)	55.13 (+15.52)	92.18 (+4.37)

7.16% and 15.52% on PLT for $LCov@2$ and $LCov@5$, respectively. We indicate that the key reason for this is that the LLMs based on its memorization ability can generate test cases that cover more lines in PLT, but they struggle to generate test cases that cover the lines in ULT⁶.

Answer to RQ1.2: LLMs struggle more with line coverage on ULT compared to other datasets. The average $LCov@5$ on ULT is 45.10%, while it is 55.13% on PLT and 92.18% on TestEval.

5.1.3 RQ1.3: Branch Coverage of LLM-generated test cases. Next, we analyze the branch coverage of the test cases generated by the LLMs on both ULT and other datasets. Compared to line coverage, branch coverage is a more complex metric that evaluates how well the test cases exercise different execution paths in the code, especially for functions with high cyclomatic complexity. The evaluation results are shown in Tab. 5, which reveal that the branch coverage of LLM-generated test cases on ULT is significantly lower than on other datasets. For example, the average branch coverage for a single generated test ($BCov@1$) across all models is 17.48% on ULT, compared to 20.20% and 69.77% on PLT and TestEval, which indicates that the functions in ULT are inherently more difficult to cover, as they often involve intricate control flows and multiple branches. Even with multiple test cases, the branch coverage on ULT remains lower than on PLT and TestEval. For instance, the average $BCov@5$ on ULT is 30.22%, while it reaches 40.07% and 82.04% on PLT and TestEval. This suggests that even with multiple attempts, LLMs struggle to explore the control flow of the complex, real-world functions in ULT as effectively as they do for the simpler problems in other datasets.

Answer to RQ1.3: LLMs struggle more with branch coverage on ULT compared to other datasets. The average $BCov@5$ on ULT is 30.22%, while it is 40.07% and 82.04% on PLT and TestEval, indicating that the functions in ULT are inherently more difficult to cover due to their complex control flows.

⁶We indicate the key reason for the lower improvement on TestEval are due to most of the lines in the function have been covered. Then, LLMs can only achieve lower improvement in line coverage for more test cases

Table 5. RQ1.3: Branch coverage of LLM-generated test cases in TestEval and ULT. $BCov@k$ represents the percentage of possible execution branches (e.g., from “if” or “while” statements) that are traversed by the first K queries generated test cases. The values in parentheses indicate the improvement over the $BCov@1$. **Insight:** Even with 5 queries ($BCov@5$), coverage on ULT (30.22%) remains significantly lower than on TestEval (82.04%), emphasizing the challenge of traversing the diverse execution branches in ULT.

Model_Name	$BCov@1$			$BCov@2$			$BCov@5$		
	ULT	PLT	TestEval	ULT	PLT	TestEval	ULT	PLT	TestEval
CodeLlama-7b-Instruct-hf	15.55	18.08	61.93	19.20 (+3.65)	24.54 (+6.46)	66.29 (+4.36)	22.33 (+6.77)	29.53 (+11.46)	70.73 (+8.80)
Seed-Coder-8B-Instruct	21.19	23.48	76.52	28.01 (+6.82)	32.94 (+9.46)	83.03 (+6.51)	35.97 (+14.77)	45.97 (+22.49)	89.21 (+12.69)
gemma-3-4b-it	15.83	19.04	48.28	21.21 (+5.38)	27.37 (+8.32)	54.07 (+5.78)	28.52 (+12.69)	39.64 (+20.60)	58.77 (+10.49)
gemma-3-12b-it	20.13	22.55	72.68	27.51 (+7.38)	33.26 (+10.71)	82.30 (+9.62)	36.96 (+16.83)	48.20 (+25.65)	90.88 (+18.20)
gemma-3-27b-it	20.82	23.83	74.97	28.85 (+8.03)	34.38 (+10.54)	83.93 (+8.96)	39.29 (+18.47)	49.64 (+25.81)	90.89 (+15.92)
Qwen2.5-Coder-7B-Instruct	19.60	22.03	74.18	26.57 (+6.97)	31.87 (+9.84)	82.92 (+8.74)	35.75 (+16.15)	46.62 (+24.59)	90.01 (+15.83)
Qwen2.5-Coder-14B-Instruct	20.31	22.70	76.82	27.47 (+7.16)	32.56 (+9.87)	83.80 (+6.98)	37.01 (+16.70)	48.17 (+25.47)	90.92 (+14.10)
Qwen2.5-Coder-32B-Instruct	20.40	23.12	75.47	28.53 (+8.13)	34.17 (+11.05)	84.32 (+8.85)	37.57 (+17.17)	49.01 (+25.89)	91.26 (+15.79)
deepseek-coder-1.3b-instruct	12.34	15.58	66.18	15.19 (+2.85)	20.93 (+5.34)	71.58 (+5.40)	15.62 (+3.28)	21.79 (+6.21)	71.81 (+5.63)
deepseek-coder-6.7b-instruct	12.25	16.07	73.62	16.63 (+4.39)	24.08 (+8.01)	80.62 (+7.00)	22.16 (+9.91)	34.97 (+18.89)	86.79 (+13.18)
deepseek-coder-33b-instruct	14.77	17.57	75.76	19.74 (+4.96)	25.21 (+7.64)	81.88 (+6.12)	25.13 (+10.36)	34.55 (+16.97)	86.37 (+10.61)
Phi-4-mini-instruct	16.56	18.28	60.82	21.37 (+4.81)	25.30 (+7.01)	63.66 (+2.84)	26.30 (+9.74)	32.80 (+14.51)	66.82 (+6.00)
Overall	17.48	20.20	69.77	23.36 (+5.88)	28.88 (+8.69)	76.53 (+6.76)	30.22 (+12.74)	40.07 (+19.88)	82.04 (+12.27)

5.1.4 RQ1.4: *Mutation Score of LLM-generated test cases.* Finally, we evaluate the mutation score of the test cases generated by the LLMs on ULT, PLT, and TestEval⁷. The mutation score is a measure of how well the test cases can detect faults in the code by introducing small changes (mutants) and checking if the test cases can catch them. As shown in Tab. 6, the performance of LLMs on ULT is notably weaker than on the other benchmarks. On average, the $Mut@5$ score for ULT is only 40.21%, significantly lower than the 50.80% on PLT and 49.69% on TestEval. This disparity indicates that tests generated for ULT’s complex, real-world functions are less effective at identifying potential faults compared to those for the simpler or potentially memorized problems in the other datasets.

While generating more tests improves the score across all datasets, the low starting point for ULT highlights the underlying challenge. The average $Mut@1$ score for ULT is a mere 20.32%, compared to 28.93% for PLT and a much higher 40.35% for TestEval. Although this low base allows for a large absolute improvement to 40.21% at $Mut@5$ (+19.89%), the final score still lags considerably behind the other benchmarks. This pattern suggests that while multiple attempts can enhance test quality on ULT, LLMs struggle to generate a single, high-quality test, and the cumulative result remains less effective than for simpler problems.

Answer to RQ1.4: The tests generated by LLMs have lower mutation scores on ULT compared to TestEval and PLT. The average $Mut@5$ on ULT is 40.21%, while it is 50.80% on PLT and 49.69% on TestEval.

5.2 RQ2: To what extent does the cyclomatic complexity of functions in ULT influence LLM performance compared to other benchmarks?

One of the key design goals of ULT is to provide a more rigorous evaluation of LLMs by presenting them with tasks that mirror the complexity of real-world software. To investigate whether this design goal was achieved, our RQ2 examines the extent to which code complexity impacts the performance of LLM-driven test generation. We hypothesize that the higher and more diverse complexity inherent in ULT functions poses a substantially greater

⁷Due to the size of PLT is very large, which requires substantial computational resources for mutation testing evaluation, we then randomly sample a subset (3,909 functions in ULT + 1,000 functions from other functions in PLT) for the mutation testing

Table 6. RQ1.4: Mutation score of LLM-generated test cases in TestEval and ULT. $Mut@k$ represents the percentage of mutants that are killed by the first K queries generated test cases. The values in parentheses indicate the improvement over the $Mut@1$. **Insight:** The fault detection capability on ULT (Avg 40.21% at $k = 5$) is notably lower than other benchmarks, suggesting that tests generated for complex, uncontaminated code are less effective at finding bugs.

Model Name	$Mut@1$			$Mut@2$			$Mut@5$		
	ULT	PLT	TestEval	ULT	PLT	TestEval	ULT	PLT	TestEval
Phi-4-mini-instruct	23.84	28.98	47.18	29.13 (+5.29)	37.28 (+8.30)	51.27 (+4.09)	36.00 (+12.16)	47.83 (+18.85)	54.49 (+7.31)
CodeLlama-7b-Instruct-hf	19.56	26.17	22.26	25.91 (+6.35)	34.55 (+8.38)	23.02 (+0.76)	29.02 (+9.47)	41.31 (+15.14)	24.70 (+2.44)
Seed-Coder-8B-Instruct	25.66	26.18	56.71	36.96 (+11.30)	39.61 (+13.43)	64.99 (+8.28)	52.04 (+26.39)	57.30 (+31.12)	71.25 (+14.54)
gemma-3-4b-it	10.56	18.89	41.80	13.62 (+3.06)	27.69 (+8.80)	46.28 (+4.48)	20.92 (+10.36)	41.38 (+22.50)	49.01 (+7.21)
gemma-3-12b-it	26.24	24.63	56.71	38.33 (+12.09)	38.50 (+13.87)	64.99 (+8.28)	52.68 (+26.45)	57.74 (+33.11)	71.25 (+14.54)
gemma-3-27b-it	21.21	27.57	34.18	28.36 (+7.14)	37.73 (+10.15)	36.68 (+2.50)	39.91 (+18.69)	55.00 (+27.43)	40.35 (+6.17)
Qwen2.5-Coder-7B-Instruct	16.60	46.06	23.53	46.35 (+29.75)	55.11 (+9.05)	26.96 (+3.43)	50.39 (+33.79)	58.86 (+12.81)	30.91 (+7.38)
Qwen2.5-Coder-14B-Instruct	22.82	36.36	41.97	40.70 (+17.88)	47.08 (+10.72)	47.31 (+5.34)	48.84 (+26.01)	55.81 (+19.44)	52.72 (+10.75)
Qwen2.5-Coder-32B-Instruct	24.62	40.99	56.71	50.23 (+25.61)	51.00 (+10.01)	64.99 (+8.28)	56.33 (+31.71)	59.57 (+18.58)	71.25 (+14.54)
deepseek-coder-1.3b-instruct	17.65	26.56	30.16	24.04 (+6.39)	35.83 (+9.27)	34.99 (+4.83)	24.47 (+6.82)	36.75 (+10.19)	41.04 (+10.88)
deepseek-coder-6.7b-instruct	16.91	19.95	36.46	25.87 (+8.95)	32.22 (+12.27)	44.19 (+7.73)	37.00 (+20.09)	50.02 (+30.06)	51.04 (+14.58)
deepseek-coder-33b-instruct	18.13	24.86	36.55	25.90 (+7.77)	34.43 (+9.57)	38.26 (+1.71)	34.88 (+16.74)	48.02 (+23.16)	38.26 (+1.71)
Average	20.32	28.93	40.35	32.12 (+11.80)	39.25 (+10.32)	45.33 (+4.98)	40.21 (+19.89)	50.80 (+21.86)	49.69 (+9.34)

challenge to LLMs compared to the more constrained, algorithmic tasks found in prior benchmarks like TestEval. To quantify this, we use Cyclomatic Complexity, a well-established metric that measures the number of linearly independent paths through a program's source code, as a proxy for structural complexity. We then correlate this metric with the achieved Branch Coverage ($BCov@k$) of the generated test cases across various LLMs. The evaluation results are shown in Fig. 1, which illustrates the relationship between cyclomatic complexity and branch coverage for both ULT and TestEval across multiple LLMs⁸.

As shown in Fig. 1, the first step in our analysis is to compare the distribution of cyclomatic complexity between ULT and TestEval. The x-axis represents the cyclomatic complexity of the function under test, grouped into bins of 10 for ULT and 5 for TestEval to accommodate the broader range of complexities in ULT. We can observe that ULT features a significantly higher and broader complexity distribution compared to TestEval. For example, the cyclomatic complexity of ULT tasks spans a wide range from 10.0 to 82.0, with a mean complexity of 14.87. In contrast, TestEval contains tasks that are far more constrained, with its complexity concentrated in a narrow band from 10.0 to 40.0 and a lower mean of 12.35. The cyclomatic complexity of ULT and TestEval explain the stark performance differences observed in RQ1. We also provide the cyclomatic complexity distribution of ULT and TestGenEval in Section 6.3, which also shows that ULT has a wider distribution of cyclomatic complexity than TestGenEval.

Next, we analyze the performance of LLMs on ULT and TestEval at equivalent levels of cyclomatic complexity. This allows us to isolate the effect of task complexity from the inherent differences in task nature between the two benchmarks. We focus on the cyclomatic complexity range of $[10, 20)$, where both benchmarks have a sufficient number of tasks, and we can make a direct comparison. The evaluation results are shown in Fig. 1, where we can see a significant performance gap between the two benchmarks. For instance, the average $BCov@5$ for CodeLlama-7B-Instruct-hf on TestEval tasks with cyclomatic complexity between 10 and 20 is close to 70%, while the average $BCov@5$ for the same model on ULT tasks in the same complexity range is lower than 30%, which means that there is about 40% performance degradation for the same complexity level.

The observation that LLMs perform significantly worse on ULT compared to TestEval, even within the same complexity bin, can be explained by differences in how complexity is defined and distributed. TestEval tasks often

⁸We also provide the comparison of cyclomatic complexity distributions between ULT and TestEval in Fig. 2.

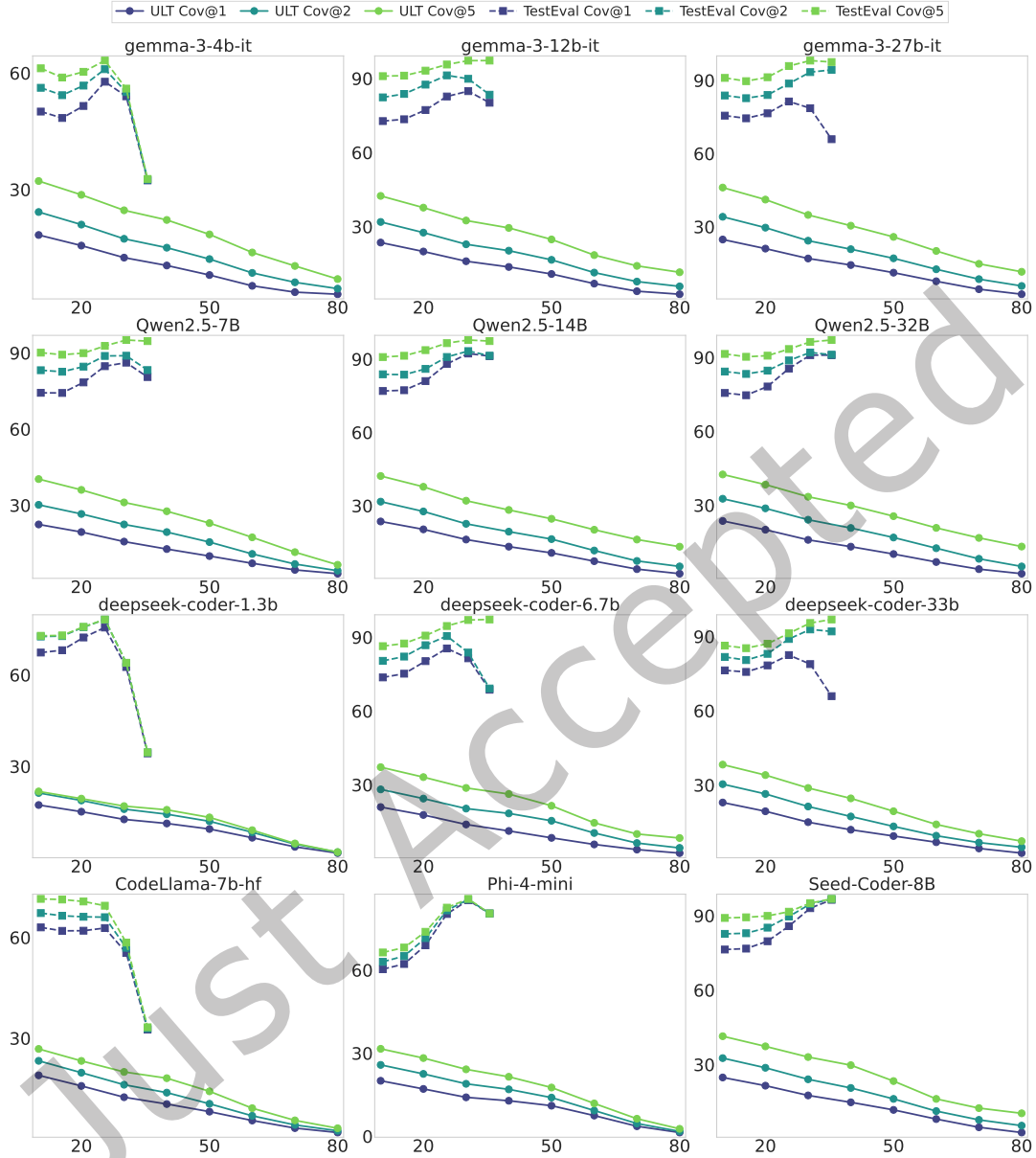


Fig. 1. Branch Coverage (Y-axis) as a function of Cyclomatic Complexity (X-axis) for ULT and TestEval across different LLMs. The X-axis represents complexity bins (e.g., [10, 20]), and the Y-axis represents the percentage of branches covered.

involve class-level solutions where the reported cyclomatic complexity is the sum of complexities across multiple smaller methods (e.g., five helper functions with CC=2 sum to CC=10). In such algorithmic problems, a single test input often naturally propagates through multiple helper functions, achieving high coverage easily. In contrast, ULT focuses on individual, standalone functions where a CC of 10 represents a single unit with dense, nested

control flow logic. Consequently, achieving high coverage on a ULT task requires generating inputs that navigate a much more intricate internal structure than a TestEval task of nominally equivalent complexity.

Next, we observe a clear and consistent negative correlation between increasing complexity and test generation performance within the ULT dataset itself. As shown in Fig. 1, for all models, the trend line for ULT exhibits a steady decline as cyclomatic complexity grows. For example, the average branch coverage ($BCov@5$) begins at 42.95% for the simplest functions (complexity [10, 20)) for Qwen2.5-Coder-7B-Instruct. The performance then systematically degrades, dropping to 37.48% for the [20, 30) bin, and falling to a mere 2.40% for highly complex functions in the [80, 90) bin.

However, we must nuance this observation. This steep decline in the high-complexity regime should not be interpreted solely as the model “struggling” to understand code. Instead, it largely reflects *structural coverage saturation*: with a fixed budget of $k = 5$ generated tests, it is often mathematically impossible to achieve high coverage on functions with such dense branching (e.g., 80+ branches). Thus, the near-zero scores in the tail of the distribution reflect the physical limits of the test budget as much as model capability.

Conversely, unlike the clear trends observed in ULT, the trend lines for TestEval exhibit inconsistent performance behaviors. For example, for the [10, 15) bin, the average $BCov@5$ for Qwen2.5-Coder-7B-Instruct is around 91.25%, but this drops to only 87.90% for the [15, 20) bin, and then further increases to 91.67% for the [25, 30) bin.

Answer to RQ2: ULT provides a significantly higher and broader distribution of cyclomatic complexity compared to TestEval, with complexities ranging from 10.0 to 82.0 and a mean of 14.87. When comparing performance at equivalent complexity levels, LLMs show a substantial performance drop on ULT, indicating that ULT’s tasks are inherently more complex and challenging. Additionally, while performance degrades as complexity increases, extreme drops in high-complexity regions are partly attributable to structural coverage saturation, where the fixed test budget ($k = 5$) is mathematically insufficient to cover the vast number of program paths.

5.3 RQ3: How does data contamination affect the assessment of test case generation?

5.3.1 RQ3.1: How does ULT compare to PLT for the same Cyclomatic Complexity? We’ve compared the overall performance of ULT and PLT in RQ1. To further investigate how data contamination affects the assessment result of test case generation, we control Cyclomatic Complexity and provide further analysis. Specifically, we provide the $BCov@k$ performance of the 12 LLMs on both ULT and PLT, as shown in Fig. 2⁹.

As shown in Fig. 2, we can observe that for all models, the branch coverage achieved by the generated test cases on PLT is significantly higher than on ULT, even when the cyclomatic complexity is equivalent. For example, for Qwen2.5-Coder-7B-Instruct, the average $BCov@5$ on functions with cyclomatic complexity between 10 and 20 is over 55% on PLT, while it is only around 40% on ULT. With the increases in cyclomatic complexity, the performance gap still exists, and the branch coverage on PLT is consistently higher than on ULT. PLT and ULT were collected in exactly the same way by the same researchers using the same process, with only one difference: whether or not the test cases were available to LLMs for training. Since the only difference is whether the unit tests were leaked, the most likely explanation for the performance gap is undoubtedly that the LLMs have used the leaked unit tests in pre-training. This not only inevitably leads to artificially inflated performance metric values free LM performance on the leaked data set, it allows us to *empirically measure* the effect size of leakage on LLM performance. By contrast, the lower branch coverage on ULT suggests that the functions are more challenging and require genuine reasoning capabilities to generate effective test cases, rather than relying on memorization of

⁹To improve the readability, we focus on the data points with cyclomatic complexity in 99.5% percentile of the cyclomatic complexity distribution of PLT, which is [10, 90), same as the complexity range of ULT.

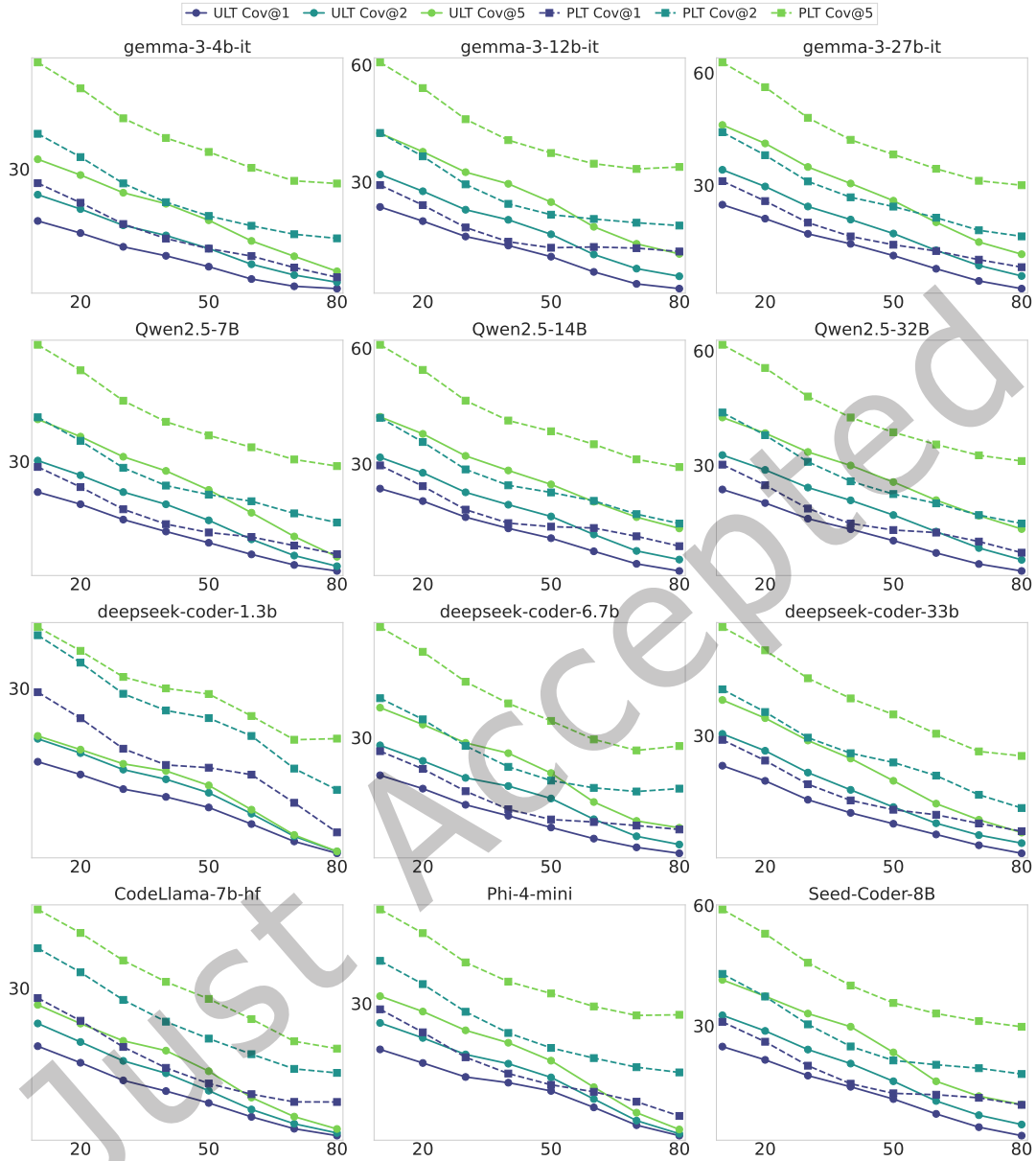


Fig. 2. Branch Coverage (Y-axis) as a function of Cyclomatic Complexity (X-axis) for ULT and PLT across different LLMs. The X-axis represents complexity bins (e.g., $[10, 20)$), and the Y-axis represents the percentage of branches covered.

previously seen test cases. This confirms that our decontamination process was successful in creating a benchmark that genuinely challenges the models' reasoning abilities rather than their capacity for memorization.

Table 7. Branch Coverage on matched subsets of ULT (ULT) and PLT (PLT), controlled for Cyclomatic Complexity, documentation, and query count. We sample 50 tasks per complexity level (11-15). For each task, we generate N test cases where N equals the cyclomatic complexity (e.g., 15 tests for level 15). The results show that PLT consistently outperforms ULT across all levels, demonstrating that data leakage—not structural constraints—is the primary driver of the performance gap.

Dataset	Qwen2.5-Coder-32B					DeepSeek-Coder-33B				
	11	12	13	14	15	11	12	13	14	15
ULT	60.92	54.60	58.08	60.00	60.24	49.96	54.68	52.08	45.68	38.16
PLT	67.02	73.64	63.78	62.30	66.54	67.98	64.62	58.44	57.20	53.52

Answer to RQ3.1: ULT has significantly lower branch coverage achieved by LLMs on ULT compared to PLT for functions of equivalent cyclomatic complexity.

5.3.2 RQ3.2: Can performance differences be explained by structural constraints? A potential threat to the validity of the comparison in RQ3.1 is the influence of structural constraints on coverage. One might argue that the reduced coverage observed on ULT is not evidence of LLMs struggling with reasoning, but rather a mathematical consequence of the branching structure. For instance, it is structurally impossible to achieve high coverage on a function with 50 branches when the experiment is limited to generating only 5 test cases. To rule out this confounding factor and isolate the impact of data leakage, we conducted a rigorous controlled analysis using matched subsets of tasks from both datasets. We explicitly controlled for three key variables to ensure a fair comparison. First, we grouped functions into specific, narrow cyclomatic complexity bins (Levels 11–15). Second, we filtered both datasets to select only functions that possess docstrings, ensuring that documentation quality was consistent across benchmarks. Third, and most crucially, we dynamically scaled the number of generated test cases to match the cyclomatic complexity of the function (e.g., generating 11 test cases for a function with complexity level 11). This design ensures that the model has the theoretical capacity to visit all branches, thereby removing the “structural impossibility” constraint.

The results are presented in Tab. 7. We observe that even when the structural constraint is removed by allowing sufficient test generation attempts, the performance gap remains significant. For example, for functions with complexity 11, Qwen2.5-Coder-32B achieves **67.02%** branch coverage on PLT compared to **60.92%** on ULT. The gap is even more pronounced for DeepSeek-Coder-33B at complexity level 15, where it scores **53.52%** on PLT versus just **38.16%** on ULT. Since the query budget is no longer a limiting factor, this analysis confirms that the difficulty of ULT stems from the lack of memorized solutions (unleaked nature) rather than the structural complexity itself.

Answer to RQ3.2: Even when controlling for structural constraints by scaling the number of generated tests to match cyclomatic complexity, LLMs perform significantly better on PLT than ULT. This confirms that the performance gap is primarily driven by data leakage.

5.3.3 RQ3.3: Can performance differences be explained by confounding factors like documentation quality? A potential threat to the validity of comparing ULT and PLT is the presence of confounding factors. Specifically, repositories that contain tests (the source of PLT) might be better maintained, feature higher-quality documentation, or follow cleaner coding standards than those without tests (the source of ULT). It is possible that LLMs perform better on PLT not because of data leakage, but because the code itself is easier to understand due to these factors. To

Table 8. Comparative Accuracy ($Pass@1$) on matched subsets of ULT and PLT, controlled for both Cyclomatic Complexity and the presence of documentation. This isolates leakage as the primary variable influencing performance.

Dataset	Qwen2.5-Coder-32B					DeepSeek-Coder-33B				
	11	12	13	14	15	11	12	13	14	15
ULT	20.90	18.41	18.41	16.92	15.92	15.92	15.42	12.94	11.94	11.44
PLT	70.65	69.15	68.66	65.67	58.21	61.19	61.19	58.71	51.24	49.25

rule out this possibility and isolate the impact of data leakage, we conducted a controlled analysis using matched subsets of tasks from both datasets. We controlled for two key variables. First, we grouped functions into specific, narrow Cyclomatic Complexity bins (11-15) to ensure structural difficulty is identical. Then, we filtered both datasets to select only functions that possess docstrings. This serves as a proxy for code quality and maintenance, ensuring we are not comparing well-documented code in PLT against poorly documented code in ULT.

The evaluation results are shown in Tab. 8, which reveal the massive performance gap persists despite these controls. For example, for functions with complexity 11 that include documentation, Qwen2.5-Coder-32B achieves **70.65%** accuracy on PLT but only **20.90%** on ULT. Since both complexity and documentation quality are standardized in this comparison, this $\sim 50\%$ gap cannot be attributed to the code being inherently “easier” or “better documented” in the leaked set. Instead, this result provides strong empirical evidence that the presence of these tests in the pre-training data (leakage) is the dominant factor driving the inflated performance on PLT.

Answer to RQ3.3: Even when controlling for confounding factors such as cyclomatic complexity and documentation quality, LLMs perform significantly better on PLT than ULT. This confirms that the performance gap is primarily driven by data leakage rather than differences in code quality.

5.3.4 RQ3.4: What is the correlation between test generation performance and code generation performance?

To further investigate the effectiveness of ULT in assessing LLMs’ test generation capabilities, we analyze the correlation between test generation performance on ULT, TestEval, and PLT with LLMs’ code generation performance on BigCodeBench [40]¹⁰. In our evaluation, we evaluate the relationship using three correlation metrics: Pearson (r), Spearman rank correlation (ρ), and Kendall tau rank-based correlation (τ). The evaluation results are shown in Fig. 3. We include rank-based metrics in addition to the standard Pearson metric because there is no intrinsic reason to assume that the relationship between test generation quality and general coding performance must be strictly linear.

We begin by analyzing the correlation regarding test generation accuracy ($Pass@1$). We observe a very strong and statistically significant rank correlation for ULT (Spearman $\rho = 0.87$, $p < 0.001$; Kendall $\tau = 0.72$, $p = 0.001$). Interestingly, TestEval also exhibits a significant positive correlation ($\rho = 0.71$, $p = 0.010$), indicating that it generally aligns with model capabilities. However, PLT displays a considerably and noticeably lower correlation ($\rho = 0.57$), with a p-value ($p = 0.051$) that is borderline statistically insignificant, which suggests that performance on PLT is not driven by the models’ general reasoning ability. Next, we extend this analysis to code coverage metrics ($LCov@1$ and $BCov@1$) to assess the quality of the generated tests. For ULT, the correlations remain strong across both line and branch coverage (Spearman $\rho \approx 0.80$, $p \leq 0.002$). Similarly, TestEval maintains a high correlation

¹⁰We use BigCodeBench as an established proxy for an LLM’s general coding ability. It is unsuitable as a test generation benchmark for our evaluation because its primary objective is to assess code generation from a prompt, often for tasks with lower structural complexity. In contrast, our focus is on a model’s ability to comprehend existing, complex code to achieve high path coverage, a core challenge of unit testing that requires the high cyclomatic complexity curated in our benchmark.

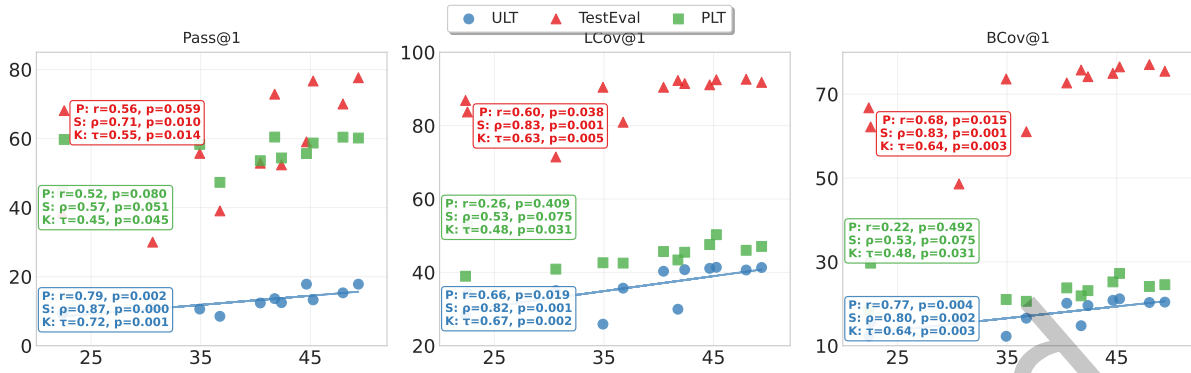


Fig. 3. Correlation between code generation performance on BigCodeBench (x-axis) and test generation performance metrics (y-axis).

for coverage metrics ($\rho \approx 0.83$). However, PLT again fails to show a strong relationship. For branch coverage, the correlation drops to $\rho = 0.53$ ($p = 0.075$), which is statistically insignificant. This reinforces the concern that high coverage on PLT is likely an artifact of data contamination rather than a valid measure of test generation capability.

The implications of these findings are significant for benchmark validity. While a high correlation does not fundamentally guarantee the absence of leakage, the *absence* of correlation is a critical warning sign. The results show that ULT meets the expected quality criteria for a valid benchmark (high rank correlation), whereas PLT fails this check. Combined with the controlled experiments in RQ3.2 and RQ3.3, we conclude that ULT provides the most scientifically valid assessment of generalization capability among the studied suites.

Answer to RQ3.4: ULT demonstrates a strong correlation with general coding ability (Spearman $\rho = 0.87$), satisfying a key quality criterion for evaluation benchmarks. In contrast, the leaked benchmark (PLT) exhibits noticeably weaker and statistically insignificant correlations ($\rho = 0.57$), suggesting that its scores are influenced by memorization rather than true coding capability.

6 Discussion

6.1 Impact of Iterative Query Count on Benchmark Suitability

To understand how the number of iterative queries (k) affects the performance of LLMs on ULT and TestEval, we conducted a detailed analysis of performance trends as k was varied from 1 to 20. The evaluation results in Fig. 4 highlight two critical and divergent patterns between ULT and TestEval.

First, we observe a significant difference in how code coverage evolves on the two benchmarks. On TestEval, both line and branch coverage exhibit rapid initial growth and then quickly converge towards a plateau. For instance, powerful models like Qwen2.5-Coder-7B-Instruct achieve nearly 95% line coverage after generating just 8-10 test cases, with subsequent tests yielding diminishing returns. This saturation indicates that the underlying functions in TestEval are structurally simple, possessing a limited number of execution paths that are easily exhausted by a small set of tests. Consequently, TestEval has a low ceiling for evaluation, making it less suitable for assessing an LLM's ability to generate a truly comprehensive and diverse test suite over extended interactions. In contrast, code coverage on ULT demonstrates a sustained growth trajectory as k increases. Even after generating 20 distinct test cases, the total line and branch coverage for all models remains below 60%. This pattern strongly

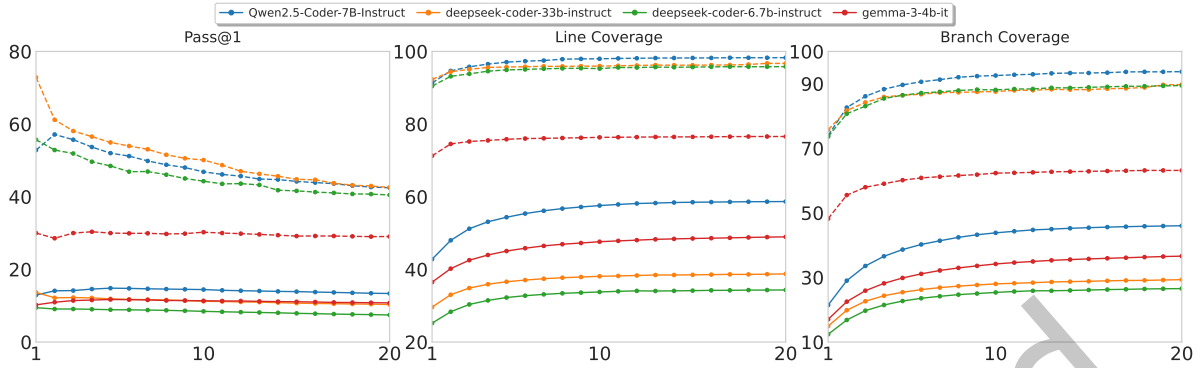


Fig. 4. Performance trends on ULT (solid lines) and TestEval (dashed lines) as the number of generated test cases (k) increases from 1 to 20. As the overhead of mutation testing for 20 times execution is very large, we do not conduct the mutation testing for this evaluation.

suggests that the functions within ULT are significantly more complex, containing a large number of independent execution paths that require continuous generation of novel test inputs to be explored. The lack of a coverage plateau provides a much longer and more discerning runway for evaluation. It allows for a clearer differentiation between models based on their ability to consistently reason about complex code and diversify their test generation strategies over time. This characteristic validates ULT as a more challenging and appropriate benchmark for rigorously measuring the test generation capabilities of advanced LLMs.

The second observation is the divergent trend in test accuracy ($Pass@k$) between the two benchmarks, as shown in Fig. 4. On ULT, the accuracy of generated tests remains relatively low but stable across all values of k . This behavior is consistent with models confronting a genuinely difficult and unseen task; their ability to produce a correct test does not significantly degrade as they are asked for more examples. Conversely, on TestEval, we observe a sharp and consistent decay in accuracy as k increases. For example, the accuracy for deepseek-coder-33b-instruct plummets from an initial 72.86% at $k = 1$ to 42.55% at $k = 20$. We hypothesize that this accuracy degradation is a direct symptom of data contamination within the TestEval benchmark. In the initial query ($k = 1$), LLMs are likely to retrieve and output a memorized solution—the “leaked” test case they have seen during pre-training. At this stage, the evaluation is primarily measuring the model’s memorization ability, resulting in an inflated accuracy score. However, as the iterative process continues, the prompt explicitly requires the LLM to generate a test case that is *different* from the ones it has already provided. This constraint forces the model to move beyond simple recall and engage in genuine test case generation. At this point, the task transitions from a test of memory to a true test of reasoning and generalization. The subsequent, significant drop in accuracy reflects the model’s actual, and much lower, capability for this more complex task. This phenomenon underscores the critical importance of our decontamination process and validates that ULT provides a more faithful and realistic assessment of an LLM’s test generation skills.

6.2 Query Strategy

In our experiments, to ensure that LLMs generate diverse test cases that can achieve high coverage, we requires LLMs to generate multiple test cases iteratively, with the instruction to produce a new test case that is “different” from the previous ones. To explore the effectiveness of this strategy, we compare it against a more simpler approach where the model is simply asked to generate a test case for each query without any additional guidance. To avoid LLM repeat its previous test cases, we adjust the temperature to 0.2, which encourages the model to

Table 9. Comparison of LLM performance with and without iterative feedback. The results are shown for two models, Qwen2.5-Coder-7B-Instruct and deepseek-coder-6.7b-instruct, under two different strategies: “w/o Feedback” and “w/ Feedback”. The “w/o Feedback” strategy uses the first query prompt asking for a different test case, while the “w/ Feedback” strategy incorporates feedback from the previous query generated test cases.

Model Name	Strategy	Test Accuracy (<i>Pass@k</i>)			Line Coverage (<i>LCov@k</i>)			Branch Coverage (<i>BCov@k</i>)		
		<i>k</i> = 1	<i>k</i> = 2	<i>k</i> = 5	<i>k</i> = 1	<i>k</i> = 2	<i>k</i> = 5	<i>k</i> = 1	<i>k</i> = 2	<i>k</i> = 5
Qwen2.5-Coder-7B-Instruct	w/o Feedback	5.45	5.26	5.08	17.04	17.58 (+0.54)	18.12 (+1.08)	9.45	10.20 (+0.75)	11.07 (+1.62)
	w/ Feedback	5.81	6.06	6.42	16.65	18.77 (+2.12)	21.32 (+4.67)	9.05	12.00 (+2.95)	15.91 (+6.86)
deepseek-coder-6.7b-instruct	w/o Feedback	4.14	3.39	2.63	9.93	10.31 (+0.38)	10.57 (+0.64)	5.17	5.70 (+0.53)	6.06 (+0.89)
	w/ Feedback	4.02	3.82	3.73	9.94	11.40 (+1.46)	12.96 (+3.02)	5.23	7.22 (+1.99)	9.52 (+4.29)

Table 10. Comparison of cyclomatic complexity distribution between ULT and TestGenEval.

Benchmark	Range	< 10	>=10	Mean	Median	Min	Max
ULT	10-82	0	100	14.87	12	10	82
TestGenEval	0-35	87.3	12.7	4.71	3	1	35

produce diverse outputs while still allowing it to generate valid test cases. The evaluation results are shown in Tab. 9, we can observe that our feedback-driven query strategy (w/ Feedback) significantly outperforms the way where LLMs are simply asked to generate different test cases (w/o Feedback) across all metrics and models. For example, for Qwen2.5-Coder-7B-Instruct, the line coverage (*LCov@5*) improved from 16.65% to 21.32% with feedback, a gain of +4.67%, while the improvement of line coverage without feedback was only +1.08%. Similarly, the branch coverage (*BCov@5*) increased from 9.05% to 15.91% with feedback, a gain of +6.86%, while the improvement without feedback was only +1.62%. This trend is consistent across both models and all metrics, indicating that the feedback-driven query strategy is significantly more effective in guiding LLMs to generate high-quality test cases that cover more lines and branches of the code under test.

6.3 Comparison with TestGenEval

To further validate the complexity and realism of ULT, we compared its cyclomatic complexity distribution with that of TestGenEval [6], another recent benchmark for test generation. As shown in Tab. 10, we can observe that ULT has a significantly higher cyclomatic complexity range, with a mean complexity of 14.87 and a maximum of 82, compared to TestGenEval’s mean complexity of 4.71 and maximum of 35. This indicates that ULT contains more complex functions that require advanced reasoning capabilities to generate effective test cases. Furthermore, the cyclomatic complexity of ULT is concentrated in a narrower range, with 100% of its functions having a cyclomatic complexity of 10 or higher, while TestGenEval has 87.3% of its functions with a cyclomatic complexity of 9 or less. This further supports our claim that ULT provides a more challenging and realistic benchmark for function-level unit test generation, as it requires LLMs to navigate complex logic and cover multiple execution paths within individual functions, rather than simply generating tests for larger files with lower complexity.

6.4 Analysis of Generated Test Input Type

To investigate the substantial performance discrepancy between ULT and PLT (e.g., average *Pass@1* of 12.69% vs 48.42%), we conducted a granular analysis of the test inputs generated by a representative model, DeepSeek-Coder-33B-Instruct. Our objective was to determine whether the low performance on ULT is driven by a disproportionately high frequency of complex input types (e.g., complex objects or mocks) or if it reflects a

Table 11. Distribution of input types generated by DeepSeek-Coder-33B and the corresponding Accuracy (*Pass@1*) on ULT vs. PLT. The “Tasks” column indicates the percentage of generated tests falling into that category.

Input Type	ULT		PLT	
	Tasks (%)	Accuracy (%)	Tasks (%)	Accuracy (%)
Primitive	27.01	21.50	39.09	67.96
Structured	24.74	19.54	31.52	59.90
Object/Complex	8.08	4.11	3.56	10.77
Others	40.16	6.62	25.83	11.91

broader failure in reasoning. We categorized the generated test inputs into four distinct types: Primitive (simple scalars), Structured (native collections), Object/Complex (custom classes or state-dependent variables), and Others (cases involving syntax errors or missing calls). This categorization not only helps in analyzing leakage but also demonstrates that our constructed dataset effectively evaluates LLMs’ test generation capabilities across a diverse spectrum of test types, ranging from simple value checks to complex state management.

The comparative results between ULT and PLT are presented in Tab. 11. When analyzing the distribution of input types, we observe that the requirements are relatively consistent across both benchmarks. Primitive and Structured inputs constitute the majority of tasks in both datasets, accounting for approximately 52% of tasks in ULT and 70% in PLT. This similarity suggests that the functions in ULT do not impose radically different or unfair structural requirements compared to the real-world code found in PLT. While ULT does have a slightly higher proportion of “Object/Complex” tasks (8.08% vs 3.56%) and “Others” (40.16% vs 25.83%), the fundamental composition remains comparable.

However, a stark contrast appears when examining the accuracy across these categories. PLT exhibits significantly higher accuracy across all input types, not just the simple ones. For instance, on Primitive inputs, the accuracy on PLT (67.96%) is more than three times higher than on ULT (21.50%). Similarly, for Structured inputs, PLT (59.90%) vastly outperforms ULT (19.54%). This uniform performance degradation on ULT indicates that the performance gap is not attributable to ULT containing an excess of complex types. Instead, the fact that LLMs fail on ULT even when the required inputs are simple primitives provides compelling evidence of data leakage in PLT. It suggests that the high performance on PLT is likely driven by the model’s ability to recall memorized test patterns for known functions, whereas ULT successfully isolates and exposes the model’s limitations in genuine, unseen test generation reasoning.

6.5 Case Study of Data Leakage

To empirically demonstrate that the performance disparity between ULT and PLT is driven by data leakage, we present a case example using StarCoder2, a model pre-trained on The Stack V2. We selected `check_color_position`, a function from PLT that requires a distinct input structure comprising a list of eight 3D coordinate tuples. We hypothesized that if the model relies on memorization, it should be capable of reconstructing this arbitrary sequence from a truncated prompt. As illustrated below, when presented with a partial input containing only the first three tuples, StarCoder2 successfully auto-completed the remaining five tuples in the exact, arbitrary order present in the original repository.

- **Ground Truth Input:**

```
1 check_color_position([(1, 1, 1)], [(1, 1, -1)], [(1, -1, 1)], [(1, -1, -1)], [(-1, 1, 1)], [(-1, 1, -1)], [(-1, -1, 1)], [(-1, -1, -1)], 1)
```

- **Truncated Prompt (Input to Model):**

```
1 check_color_position([[1, 1, 1]], [[1, 1, -1]], [[1, -1, 1]],
```

- **Model Completion (StarCoder2):**

```
1 [[1, -1, -1]], [[-1, 1, 1]], [[-1, 1, -1]], [[-1, -1, 1]], [[-1, -1, -1]], 1)
```

The model’s ability to reproduce this exact sequence provides compelling evidence of data leakage. Given the arbitrary ordering and specific formatting of the coordinates, the probability of generating this precise string through chance or logical inference is negligible. This observation confirms that the superior performance on PLT is largely attributable to the retrieval of memorized solutions, whereas the lower scores on ULT more accurately reflect the true reasoning limitations of models when they cannot access ground truth examples from their training data.

6.6 Avoiding Future Data Contamination

A critical challenge for any contemporary benchmark is ensuring its long-term viability in an era where public data is continuously scraped for training next-generation LLMs. This process of data contamination or leakage, where a model is inadvertently trained on the benchmark it is meant to be evaluated against, can invalidate results and obscure true scientific progress. We have designed the release and evaluation process for ULT specifically to mitigate this threat.

Our primary strategy is to separate the benchmark’s problems from its solutions. We publicly release the curated set of 3,909 Python functions that serve as the evaluation problems. However, we deliberately withhold the ground-truth test suites that we created for our internal validation. These reference tests are not required for evaluation. Instead, we provide a self-contained evaluation script. Researchers use this script to evaluate the tests generated by their models; the script dynamically executes the generated tests against the functions-under-test and calculates metrics, such as line and branch coverage, without ever exposing a static set of correct solutions that could be crawled and memorized by future models. This approach ensures that ULT remains a robust test of generation capability, not memorization.

This design also addresses the secondary threat of adversarial contamination, where a user might generate their own high-quality tests and add them to public corpora. While this is impossible to prevent entirely, our approach makes such contamination significantly more difficult and less direct than if we had provided a canonical golden set of tests ourselves. Furthermore, to foster a community-wide commitment to the benchmark’s integrity, the license for ULT will explicitly discourage users from making generated test suites publicly available in a manner that facilitates web crawling. We believe establishing this community norm is crucial for preserving the benchmark’s value, balancing the principles of open science and replicability with the practical need for contamination-resistant evaluation.

7 Threats to Validity

7.1 Internal Validity

Internal validity concerns potential confounding factors within our experimental setup that could influence the observed outcomes. A primary threat pertains to the reproducibility and determinism of the test cases generated by LLMs. The inherently stochastic nature of some decoding strategies could lead to variability in results, making it difficult to attribute performance differences solely to model capabilities. To mitigate this, we employed a greedy decoding strategy by setting the temperature parameter to 0 for all experiments, as detailed in Section 4.3. This approach significantly minimizes randomness, ensuring that the generated outputs are as deterministic as possible and primarily reflect the model’s core reasoning abilities rather than sampling artifacts. Another potential threat is the reliability of our test execution and evaluation environment. Inconsistencies in the environment,

such as differing library versions or system configurations, could lead to spurious test failures or inaccurate coverage measurements. To address this, all generated test cases were executed within a unified and standardized Docker environment. This containerized setup guarantees that every test is run against the exact same version of the Python interpreter and dependent libraries, thereby ensuring the consistency and comparability of our results across all models and benchmarks.

Finally, a significant threat arises from the observational nature of our study regarding data leakage. We attribute the performance gap between ULT and PLT primarily to test case leakage in the training data. However, because we evaluate third-party LLMs, we cannot causally intervene in their pre-training process (e.g., by retraining them on datasets with specific tests excluded). Consequently, confounding variables may contribute to the performance difference. Indeed, our own analysis in Tab. 11 reveals observable structural differences between the two datasets: for instance, ULT contains a notably higher proportion of Object/Complex input types (8.08% vs. 3.56%), a category associated with substantially lower accuracy on both benchmarks.

While we attempted to mitigate this by computationally controlling for complexity and documentation quality, we acknowledge that we cannot fully rule out the influence of other latent factors associated with well-tested code in the training corpora.

7.2 External Validity

External validity relates to the generalizability of our findings beyond the specific context of this study. One threat is the representativeness of our benchmark. Although ULT is constructed from a large corpus of real-world Python functions from The Stack v2 and curated based on specific criteria like cyclomatic complexity and test case decontamination, it may not encompass the full spectrum of programming paradigms, application domains, or coding styles found in all software projects. Therefore, while our results provide strong evidence regarding complex, self-contained functions, caution should be exercised when generalizing them to other types of software, such as large-scale enterprise systems or highly specialized domains. Moreover, our focus on isolated function-level generation limits generalizability to system-level testing. Real-world applications often involve complex state management and cross-module interactions. By design, ULT mitigates these factors to purely assess algorithmic reasoning. Consequently, our results may not reflect performance in integration testing scenarios where managing external dependencies and environmental context is the primary challenge. Another threat to external validity is our focus on a single programming language, Python. The capabilities of LLMs in code understanding and generation can vary across different languages due to differences in syntax, semantics, and representation in their training data. The findings and performance gaps observed in this study for Python may not directly translate to other languages like Java, C++, or JavaScript. Finally, our evaluation is limited to the specific set of 12 LLMs listed in Tab. 1. The field of LLMs is evolving at an exceptionally rapid pace, with new and more powerful models being released frequently. While our selection represents a broad and diverse snapshot of the current state-of-the-art, our specific findings may not be generalizable to future, more advanced models that may overcome some of the challenges identified in this work. Related to this rapid evolution is the longevity of the benchmark itself. As LLM developers continuously scrape open-source repositories for training data, there is an inherent risk that the functions currently isolated in ULT will eventually be included in future training corpora. This implies that static benchmarks like ULT may suffer from “contamination creep” over time, necessitating periodic updates or dynamic curation to maintain their validity as an “unleaked” evaluation standard.

7.3 Construct Validity

Construct validity examines whether our evaluation metrics and experimental design accurately measure the concepts they purport to assess, namely test generation quality and a model’s reasoning capability. A potential threat is our reliance on code coverage (line and branch) and mutation score as primary proxies for the quality of

a generated test suite. While these are standard and widely accepted metrics in software testing research, they are not perfect. High coverage does not guarantee the absence of bugs, and a test suite could kill many mutants without necessarily reflecting all aspects of real-world fault-finding effectiveness. Nonetheless, these metrics provide a quantitative, objective, and reproducible basis for comparison that is standard practice in the field. Furthermore, our reliance on cyclomatic complexity as a filter for task selection presents a validity threat. While cyclomatic complexity is a standard proxy for structural complexity, it does not always equate to conceptual or reasoning difficulty. For instance, a function with high complexity due to a long sequence of simple “if-else” checks might be easier for an LLM to process than a concise but algorithmically dense recursive function with low complexity. Therefore, while our filtering ensures structural intricacy, it may not fully capture all dimensions of code “difficulty”.

Additionally, we distinguish between “Algorithmic Complexity” (intricate internal control flows) and “System Complexity” (complex cross-module data dependencies). We acknowledge that real-world software difficulty often stems from the latter. However, ULT deliberately focuses on Algorithmic Complexity within function boundaries. We posit that evaluating an LLM’s ability to navigate complex internal logic is a necessary prerequisite step before assessing its capacity to handle the broader, intricate data-flow relationships inherent in full repository contexts. Another threat lies in our use of performance on BigCodeBench as a proxy for an LLM’s intrinsic, general-purpose coding ability in our RQ3 analysis. While BigCodeBench is a comprehensive benchmark, a model’s proficiency in general code generation may not perfectly correlate with its specialized ability to perform test generation, which requires different reasoning skills (e.g., identifying edge cases and defining assertions). However, by using a well-established, independent benchmark, we establish a reasonable baseline for a model’s fundamental reasoning capabilities, allowing us to effectively test our hypothesis regarding data contamination. Finally, the design of our iterative test generation task, where we prompt for a “new” and “distinct” test case in each round, could be a threat. The interpretation of these terms by the LLM might vary, and the prompt structure itself could influence the diversification strategy of the models. We acknowledge this as an inherent aspect of using natural language to prompt LLMs and have kept the prompt consistent across all models to ensure a fair comparison.

8 Related Work

Our research is situated within the rapidly growing field of applying LLMs to the software engineering task of automated test generation. This area has seen a surge of innovation, with researchers exploring various strategies to leverage the code and reasoning capabilities of LLMs. This section reviews the literature by first surveying the landscape of LLM-based test generation techniques and then discussing the evolution of benchmarks used to evaluate them, thereby positioning the unique contribution of ULT.

8.1 LLMs for Test Generation

Automated unit test generation has a long-standing history prior to the advent of LLMs. Traditionally, this field was dominated by Search-Based Software Testing (SBST) techniques [41], epitomized by tools like EvoSuite [42], which utilize evolutionary algorithms to maximize code coverage. While these approaches are effective at generating high-coverage test suites, they often struggle to produce readable code or capture complex semantic constraints. Recent research has thus shifted focus toward LLMs to address these limitations.

The application of LLMs to generate unit tests has evolved from initial feasibility studies to a variety of sophisticated techniques aimed at improving the quality and effectiveness of the generated test suites. These approaches can be broadly categorized by whether they focus on refining the model’s input (prompt engineering), enhancing the model’s output (post-processing and repair), or altering the generation process itself. A foundational stream of research has focused on empirically evaluating the baseline performance of LLMs and building practical tools. Studies by Siddiq et al. [43], Schäfer et al. [3], and El Haji et al. [44] provided crucial early insights into the

capabilities and limitations of models like Codex, GPT-3.5, and GitHub Copilot for generating tests in languages such as Java, JavaScript, and Python. This foundational work paved the way for IDE plugins like TestSpark [45], which integrate test generation directly into the developer’s workflow. Other work has focused on optimizing the input to the LLM to elicit higher-quality outputs. Researchers have shown that careful prompt engineering, such as providing contextual information about the code or its dependencies, can significantly improve the quality of generated tests [46, 47]. More advanced techniques employ retrieval-augmented generation, where relevant few-shot examples are dynamically retrieved from a corpus and included in the prompt to guide the model, as demonstrated by CEDAR [48]. Recognizing that LLMs often produce imperfect or incomplete tests, another major line of work focuses on enhancing or repairing the generated output. This includes hybrid approaches that combine LLMs with traditional software testing techniques. For instance, Codamosa [49] uses LLMs to generate new test inputs to help Search-Based Software Testing (SBST) escape from local optima and improve coverage. Other techniques focus on iterative refinement, where the LLM’s output is executed, and the resulting feedback (e.g., compilation errors, failed assertions) is used to prompt the model for a revised solution, a strategy employed by Testart [50] in a co-evolutionary cycle. Guidance from external analysis is also common, with researchers using static analysis [51] or feedback from mutation testing [52] to guide the LLM toward producing more effective and bug-finding tests. Although these techniques have shown promising results in test generation, there still lack a comprehensive benchmark that can effectively evaluate the true reasoning and generalization capabilities of LLMs in this context. Our work addresses this gap by introducing ULT, a benchmark specifically designed to challenge LLMs with complex, real-world functions while mitigating the risks of data contamination and ensuring a more accurate assessment of their test generation abilities.

8.2 Benchmarks for Test Generation

Concurrent with the development of new generation techniques has been the creation of benchmarks to evaluate their effectiveness. An early and influential benchmark, TestEval [5], established a foundation by proposing tasks based on competitive programming problems from LeetCode. It introduced key evaluation metrics, including coverage-oriented tasks, providing a standardized basis for comparing LLMs. However, as we demonstrated in our study, its reliance on algorithmic problems limits its representativeness of real-world software engineering challenges. Subsequent benchmarks sought to improve real-world relevance by sourcing tasks from large, open-source software projects. SWT-Bench [20] and TestGenEval [6] both derive their tasks from the SWE-Bench dataset [22]. SWT-Bench frames the task as issue reproduction, where the goal is to generate a test that fails on buggy code but passes on the fixed version. TestGenEval focuses on test file generation and completion, using execution-based metrics on code from major software repositories. Notably, TestGenEval also employs mutation analysis to assess test adequacy, leveraging the established correlation between mutants and real faults [53]. While these benchmarks marked a significant step towards realism, they introduced other confounding factors that ULT is explicitly designed to address. As discussed in our introduction and supported by our findings in RQ3, benchmarks derived from popular public repositories are susceptible to test case contamination, where models may score well by recalling solutions from their training data rather than by demonstrating genuine reasoning. Furthermore, their file- or repository-level task granularity often leads to excessively long input contexts, which can degrade LLM performance, and their tasks may not consistently feature high structural complexity. ULT differentiates itself from this prior work by simultaneously ensuring real-world relevance, mitigating test case contamination through a rigorous filtering process, controlling for complexity by filtering on cyclomatic complexity, and utilizing a function-level focus to enable a more precise assessment of an LLM’s core test generation capabilities.

9 Conclusion

In this paper, we introduce ULT, a new benchmark specifically designed for function-level unit test generation from real-world Python functions. ULT is designed to address critical limitations in existing benchmarks, such as test case data contamination and insufficient program complexity. By focusing on real-world functions with high cyclomatic complexity, ULT provides a more challenging and realistic evaluation environment for LLMs. We also provide PLT, a pair benchmarks of ULT with leaked tests designed to enable a controlled analysis of memorization versus reasoning in test generation. We conducted a comprehensive experimental evaluation involving 12 state-of-the-art LLMs, systematically comparing their performance on ULT against other benchmarks like TestEval, BigCodeBench, and PLT. Our findings reveal that ULT significantly outperforms existing benchmarks in terms of complexity and challenge, with a broader distribution of cyclomatic complexity ranging from 10.0 to 82.0 and a mean of 14.87, compared to TestEval’s concentration between 9.0 to 45.0, with a mean of 12.35. We also demonstrated that performance on ULT is substantially lower across all metrics (accuracy, line coverage, and branch coverage) compared to TestEval and PLT, confirming that its tasks are inherently more difficult. For example, test cases generated by LLMs only achieve 41.32%, 45.10%, 30.22%, and 40.21% for accuracy, statement coverage, branch coverage, and mutation score on average for all LLMs, respectively. These results are substantially lower than the corresponding metrics on TestEval (91.79%, 92.18%, 82.04%, and 49.69%) and PLT (47.07%, 55.13%, 40.07%, and 50.80%). This performance gap persists even when comparing tasks of similar cyclomatic complexity, indicating that ULT’s tasks are not only more complex but also require deeper reasoning and generalization capabilities from the models.

10 Future Work

To bridge the gap between isolated function-level benchmarks and realistic, project-level testing scenarios, we envision extending ULT in several key directions. First, we aim to enable **Context-Aware Test Generation** by including tasks that require understanding cross-module dependencies. This involves providing LLMs with repository-level context (e.g., imported class definitions and utility functions) to evaluate their ability to use Retrieval-Augmented Generation (RAG) or long-context windows for integration-aware unit testing. Second, we plan to assess **Mocking and Stubbing Capabilities**, as real-world unit testing often requires isolating the unit under test from external systems (e.g., databases, network APIs). Future iterations will specifically target functions with such dependencies to evaluate whether LLMs can correctly identify the need for and generate appropriate mocks. Finally, to address the challenge of “contamination creep”, we aim to implement **Dynamic Benchmark Maintenance** through an automated pipeline that continuously curates new, high-complexity, unleaked functions from emerging open-source repositories. This ensures the benchmark remains a valid test of generalization as LLM training corpora expand.

References

- [1] S. Anand, E. K. Burke, T. Y. Chen, J. Clark, M. B. Cohen, W. Grieskamp, M. Harman, M. J. Harrold, P. McMinn, A. Bertolino *et al.*, “An orchestrated survey of methodologies for automated software test case generation,” *Journal of systems and software*, vol. 86, no. 8, pp. 1978–2001, 2013.
- [2] W. Li, F. Le Gall, and N. Spaseski, “A survey on model-based testing tools for test case generation,” in *International Conference on Tools and Methods for Program Analysis*. Springer, 2017, pp. 77–89.
- [3] M. Schäfer, S. Nadi, A. Eghbali, and F. Tip, “An empirical evaluation of using large language models for automated unit test generation,” *IEEE Transactions on Software Engineering*, vol. 50, no. 1, pp. 85–105, 2023.
- [4] D. Huang, J. M. Zhang, M. Du, M. Harman, and H. Cui, “Rethinking the influence of source code on test case generation,” *arXiv preprint arXiv:2409.09464*, 2024.
- [5] W. Wang, C. Yang, Z. Wang, Y. Huang, Z. Chu, D. Song, L. Zhang, A. R. Chen, and L. Ma, “Testeval: Benchmarking large language models for test case generation,” *arXiv preprint arXiv:2406.04531*, 2024.
- [6] K. Jain, G. Synnaeve, and B. Rozière, “Testgeneval: A real world unit test generation and test completion benchmark,” *arXiv preprint*

- arXiv:2410.00752*, 2024.
- [7] E. Daka and G. Fraser, “A survey on unit testing practices and problems,” in *2014 IEEE 25th International Symposium on Software Reliability Engineering*. IEEE, 2014, pp. 201–211.
 - [8] H. Yu, T. Chen, J. Huang, Z. Li, D. Ran, X. Wang, Y. Li, A. Marron, D. Harel, Y. Xie *et al.*, “Decon: Detecting incorrect assertions via postconditions generated by a large language model,” *arXiv preprint arXiv:2501.02901*, 2025.
 - [9] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen, “Codet: Code generation with generated tests,” *arXiv preprint arXiv:2207.10397*, 2022.
 - [10] D. Huang, Q. Bu, Y. Qing, and H. Cui, “Codecot: Tackling code syntax errors in cot reasoning for code generation,” *arXiv preprint arXiv:2308.08784*, 2023.
 - [11] D. Huang, J. M. Zhang, M. Luck, Q. Bu, Y. Qing, and H. Cui, “Agentcoder: Multi-agent-based code generation with iterative testing and optimisation,” *arXiv preprint arXiv:2312.13010*, 2023.
 - [12] M. Du, A. T. Luu, B. Ji, Q. Liu, and S.-K. Ng, “Mercury: A code efficiency benchmark for code large language models,” *arXiv preprint arXiv:2402.07844*, 2024.
 - [13] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language agents with verbal reinforcement learning,” *Advances in Neural Information Processing Systems*, vol. 36, pp. 8634–8652, 2023.
 - [14] C. Yang, Z. Zhao, and L. Zhang, “Kernelgpt: Enhanced kernel fuzzing via large language models,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2025, pp. 560–573.
 - [15] C. S. Xia, M. Paltenghi, J. Le Tian, M. Pradel, and L. Zhang, “Universal fuzzing via large language models,” *CoRR*, 2023.
 - [16] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, “Large language models are edge-case generators: Crafting unusual programs for fuzzing deep learning libraries,” in *Proceedings of the 46th IEEE/ACM international conference on software engineering*, 2024, pp. 1–13.
 - [17] C. Yang, Y. Deng, R. Lu, J. Yao, J. Liu, R. Jabbarvand, and L. Zhang, “Whitefox: White-box compiler fuzzing empowered by large language models,” *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA2, pp. 709–735, 2024.
 - [18] Y. Deng, C. S. Xia, H. Peng, C. Yang, and L. Zhang, “Large language models are zero-shot fuzzers: Fuzzing deep-learning libraries via large language models,” in *Proceedings of the 32nd ACM SIGSOFT international symposium on software testing and analysis*, 2023, pp. 423–435.
 - [19] Y. Deng, C. S. Xia, C. Yang, S. D. Zhang, S. Yang, and L. Zhang, “Large language models are edge-case fuzzers: Testing deep learning libraries via fuzzgpt,” *arXiv preprint arXiv:2304.02014*, 2023.
 - [20] N. Mündler, M. Müller, J. He, and M. Vechev, “SwT-bench: Testing and validating real-world bug-fixes with code agents,” *Advances in Neural Information Processing Systems*, vol. 37, pp. 81 857–81 887, 2024.
 - [21] Q. Zhang, Y. Shang, C. Fang, S. Gu, J. Zhou, and Z. Chen, “Testbench: Evaluating class-level test case generation capability of large language models,” *arXiv preprint arXiv:2409.17561*, 2024.
 - [22] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” *arXiv preprint arXiv:2310.06770*, 2023.
 - [23] X. Zhou, M. Weyssow, R. Widyasari, T. Zhang, J. He, Y. Lyu, J. Chang, B. Zhang, D. Huang, and D. Lo, “Lessleak-bench: A first investigation of data leakage in llms across 83 software engineering benchmarks,” *arXiv preprint arXiv:2502.06215*, 2025.
 - [24] W. C. Ouédraogo, K. Kaboré, H. Tian, Y. Song, A. Koyuncu, J. Klein, D. Lo, and T. F. Bissyandé, “Large-scale, independent and comprehensive study of the power of llms for test case generation,” *arXiv preprint arXiv:2407.00225*, 2024.
 - [25] A. Lozhkov, R. Li, L. B. Allal, F. Cassano, J. Lamy-Poirier, N. Tazi, A. Tang, D. Pykhtar, J. Liu, Y. Wei *et al.*, “StarCoder 2 and the stack v2: The next generation,” *arXiv preprint arXiv:2402.19173*, 2024.
 - [26] I. Sommerville, “Software engineering (ed.),” *America: Pearson Education Inc*, 2011.
 - [27] G. J. Myers, C. Sandler, and T. Badgett, *The art of software testing*. John Wiley & Sons, 2011.
 - [28] P. Runeson, “A survey of unit testing practices,” *IEEE software*, vol. 23, no. 4, pp. 22–29, 2006.
 - [29] G. Meszaros, *xUnit test patterns: Refactoring test code*. Pearson Education, 2007.
 - [30] S. Shamshiri, “Automated unit test generation for evolving software,” in *Proceedings of the 2015 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 1038–1041.
 - [31] J. C. King, “Symbolic execution and program testing,” *Communications of the ACM*, vol. 19, no. 7, pp. 385–394, 1976.
 - [32] V. Chipounov, V. Kuznetsov, and G. Candea, “The s2e platform: Design, implementation, and applications,” *ACM Transactions on Computer Systems (TOCS)*, vol. 30, no. 1, pp. 1–49, 2012.
 - [33] P. McMinn, “Search-based software testing: Past, present and future,” in *2011 IEEE Fourth International Conference on Software Testing, Verification and Validation Workshops*. IEEE, 2011, pp. 153–163.
 - [34] B. P. Miller, L. Fredriksen, and B. So, “An empirical study of the reliability of unix utilities,” *Communications of the ACM*, vol. 33, no. 12, pp. 32–44, 1990.
 - [35] X. Hou, Y. Zhao, Y. Liu, Z. Yang, K. Wang, L. Li, X. Luo, D. Lo, J. Grundy, and H. Wang, “Large language models for software engineering: A systematic literature review,” *ACM Transactions on Software Engineering and Methodology*, vol. 33, no. 8, pp. 1–79, 2024.
 - [36] M. J. Shepperd, “A critique of cyclomatic complexity as a software metric,” *Software Engineering Journal*, vol. 3, no. 2, pp. 177–188, 1988.

- [37] B. Henderson-Sellers and D. Tegarden, "Clarification concerning modularization and McCabe's cyclomatic complexity," vol. 37, no. 4, pp. 92–94, 1994.
- [38] M. Papadakis, M. Kintis, J. Zhang, Y. Jia, Y. Le Traon, and M. Harman, "Mutation testing advances: an analysis and survey," in *Advances in computers*. Elsevier, 2019, vol. 112, pp. 275–378.
- [39] Y. Jia and M. Harman, "An analysis and survey of the development of mutation testing," *IEEE transactions on software engineering*, vol. 37, no. 5, pp. 649–678, 2010.
- [40] T. Y. Zhuo, M. C. Vu, J. Chim, H. Hu, W. Yu, R. Widyasari, I. N. B. Yusuf, H. Zhan, J. He, I. Paul *et al.*, "Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions," *arXiv preprint arXiv:2406.15877*, 2024.
- [41] P. McMin, "Search-based software test data generation: a survey," *Software testing, Verification and reliability*, vol. 14, no. 2, pp. 105–156, 2004.
- [42] G. Fraser and A. Arcuri, "Evosuite: automatic test suite generation for object-oriented software," in *Proceedings of the 19th ACM SIGSOFT symposium and the 13th European conference on Foundations of software engineering*, 2011, pp. 416–419.
- [43] M. L. Siddiq, J. C. Da Silva Santos, R. H. Tanvir, N. Ulfat, F. Al Rifat, and V. Carvalho Lopes, "Using large language models to generate junit tests: An empirical study," in *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*, 2024, pp. 313–322.
- [44] K. El Haji, C. Brandt, and A. Zaidman, "Using github copilot for test generation in python: An empirical study," in *Proceedings of the 5th ACM/IEEE International Conference on Automation of Software Test (AST 2024)*, 2024, pp. 45–55.
- [45] A. Sapozhnikov, M. Olsthoorn, A. Panichella, V. Kovalenko, and P. Derakhshanfar, "Testspark: Intellij idea's ultimate test generation companion," in *Proceedings of the 2024 IEEE/ACM 46th International Conference on Software Engineering: Companion Proceedings*, 2024, pp. 30–34.
- [46] P. Bareiß, B. Souza, M. d'Amorim, and M. Pradel, "Code generation tools (almost) for free? a study of few-shot, pre-trained language models on code," *arXiv preprint arXiv:2206.01335*, 2022.
- [47] D. Huang, J. M. Zhang, M. Harman, M. Du, and H. Cui, "Measuring the influence of incorrect code on test generation," *arXiv preprint arXiv:2409.09464*, 2024.
- [48] N. Nashid, M. Sintaha, and A. Mesbah, "Retrieval-based prompt selection for code-related few-shot learning," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 2450–2462.
- [49] C. Lemieux, J. P. Inala, S. K. Lahiri, and S. Sen, "Codamosa: Escaping coverage plateaus in test generation with pre-trained large language models," in *2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE)*. IEEE, 2023, pp. 919–931.
- [50] S. Gu, C. Fang, Q. Zhang, F. Tian, and Z. Chen, "Testart: Improving llm-based unit test via co-evolution of automated generation and repair iteration," *arXiv e-prints*, pp. arXiv–2408, 2024.
- [51] R. Pan, M. Kim, R. Krishna, R. Pavuluri, and S. Sinha, "Aster: Natural and multi-language unit test generation with llms," *arXiv preprint arXiv:2409.03093*, 2025.
- [52] A. M. Dakhel, A. Nikanjam, V. Majdinasab, F. Khomh, and M. C. Desmarais, "Effective test generation using pre-trained large language models and mutation testing," *Information and Software Technology*, vol. 171, p. 107468, 2024.
- [53] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, and G. Fraser, "Are mutants a valid substitute for real faults in software testing?" in *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, 2014, pp. 654–665.